

基本情報技術者試験 科目B対策

擬似言語とAIで学ぶ アルゴリズム入門

はじめての変数から、配列のトレースまで

「答えを聞く」のではなく、
「分からない場所を見つける」ためにAIを使おう。

基本情報技術者試験 科目B対策教材

はじめに

この教材は、基本情報技術者試験の科目Bで出題されるアルゴリズムとプログラミングを、プログラミング未経験者が擬似言語から学ぶための入門書です。

科目Bの問題を解くために、難しいプログラム言語を最初から覚える必要はありません。まず必要なのは、次の三つの力です。

- 1. 擬似言語を上から順番に読む力
- 2. 変数や配列の値がどう変化するかを追う力
- 3. 分からない箇所を言葉にして、質問する力

本研修では、処理の基本となる**順次・選択・繰返し**を擬似言語で学びます。各演習の解説には、処理の進み方を矢印で示すフローチャートも掲載します。コードの一行と図の一つの処理を対応させて読みましょう。

また、本書では生成AIへ質問するための例文を紹介します。AIに正解を直接聞くのではなく、考え方の整理、ヒント、トレースの確認、間違いの発見に利用します。

本研修の到達目標

- 変数・代入・配列を説明できる
- 選択処理と繰返し処理を読める
- 短い擬似言語をトレースできる
- 合計・件数・最大値を求める処理を理解できる
- AIを使って、自分で学習を続けられる

この教材の使い方

第0章は、全員が最初に学ぶ「AIの安全で有効な活用」です。第1～8章には各6問、合計48問を用意しています。第1章は★を2問、★★を4問、第2～8章は★・★★・★★★を各2問用意しています。

難易度	取り組み方
★☆☆ 基礎	一つの処理・値・条件を確認する
★★☆ 応用	複数の処理を組み合わせ、途中の値を説明する
★★★ 実践	仕様や利用場面を読み、境界や処理順も検証する

早く終わった人は上の難易度へ進み、条件や入力を変えても説明できるか確かめます。

各節は、次の順番で進みます。

1 概念を読む 何をする仕組みか理解する	2 処理をたどる 値が変わる順番を手で記録する	3 自力で解く 最初からAIに聞かない	4 AIに質問する 分からない箇所だけ尋ねる	5 解き直す AIを閉じてもう一度解く
--	---	---	--	---

大切なルール

AIの説明は、いつも正しいとは限りません。本書の解説や公式資料と照合し、自分で値を追って検証しましょう。「AIが言ったから正しい」ではなく、「自分でも同じ結果を確かめられたから正しい」と判断します。

目次

第0章 AIを学習相手にする	5
第1章 アルゴリズムと擬似言語	11
第2章 変数・データ型・代入	20
第3章 順次処理	31
第4章 選択処理	39
第5章 繰返し処理	50
第6章 配列	63
第7章 トレースの技術	73
第8章 総合演習	82
解答・解説	90
研修後の学習ロードマップ	139

第0章 AIを学習相手にする

最初に確認:AIを安全に、有効に使う

この章は演習より先に学びます。AIを使い慣れていても、安全のルールと学習に役立つ質問方法を確認しましょう。

確認すること	研修での行動
個人情報・機密情報	氏名、連絡先、顧客情報、社内資料、パスワードは入力しません。必要な例は架空の名前・数値に置き換えます。
利用する場所	講師が指定した共有ノートブックを使い、会社・研修の利用ルールを守ります。リンクを研修外へ転送しません。
回答の正しさ	AIの説明はもっともらしくても誤ることがあります。参照元を開き、自分のトレースと教材の解説で確かめます。
学習の目的	まず自分で考え、ヒントを一つずつ求めます。最後はAIを閉じて解き直します。

安全確認の例:「顧客Aさんの実際の購入履歴」ではなく、「架空の購入額が1000円の場合」として質問します。学習に必要な実データは渡しません。

0.1 AIは「正解を出す機械」ではない

生成AIへ問題をそのまま入力し、「答えを教えて」と頼めば、短時間で答えらしきものが表示されます。しかし、それだけでは自分で問題を解く力は身に付きません。

本研修では、AIを次の四つの役割で使います。

AIの役割	何をしてもらうか	使用する場面
整理役	入力・処理・出力を分ける	問題文の意味が分からない
ヒント役	次に見るべき場所を示す	解き始められない
点検役	最初に間違えた箇所を示す	自分の考えに自信がない
練習相手	難易度を調整した類題を作る	もう一問練習したい

避けたい質問

問題番号を示さずに「答えを教えてください」とだけ頼むことや、考える前に「完成した擬似言語を作って」と頼むこと

正解だけを受け取ると、どこで考えられなくなったのかが分かりません。

学習につながる質問

「最初に確認すべき変数を一つ教えてください」

「私のトレース表で、最初に間違っている行だけを指摘してください」

0.2 AIに質問する前の30秒

質問を入力する前に、次の三点を書き出します。

AIへ聞く前の整理メモ

- | | |
|------------|----------------|
| 1 分かっていること | ここに書く
----- |
| 2 分からないこと | ここに書く
----- |
| 3 自分で試したこと | ここに書く
----- |

例えば、次のように書きます。

記入例

- | | |
|------------|-------------------------------|
| 1 分かっていること | totalが合計を保存する変数であることは分かります。 |
| 2 分からないこと | 前の合計に次の数を足すとき、どの値を使うのか分かりません。 |
| 3 自分で試したこと | 最初の足し算までは紙に書きました。 |

ここまで書くと、AIへの質問が具体的になります。また、自分がどこまで理解できているかも見えるようになります。

0.3 研修中:共有ノートブックへ問題番号で質問する

今回の研修では、講師が本書のPDFを読み込ませたGemini Notebook(NotebookLM)のリンクを共有します。受講者が問題文をコピーして貼り付けたり、PDFを再アップロードしたりする必要はありません。講師から届いたリンクで対象ノートブックを開き、そのチャット欄に質問します。

1. 共有リンクを開き、研修用のノートブックであることを確認する。
2. 「問題2-Bについて質問です。」のように、章番号と問題の英字を指定する。
3. 自分が分かっていること、試したこと、分からない点を書く。

答えや完成コードをすぐに示さない設定は、講師が済ませています。各問題への質問では、その指示を繰り返す必要はありません。

AI

研修中の質問例 | 問題番号と疑問点を伝える

問題2-Bについて質問です。xにyを入れるところは分かりますが、tempが必要な理由が分かりません。

0.4 研修後:別の参考書の問題をAIへ伝える

自習では、質問したい問題をAIへ渡します。資料とAIの機能に合わせ、次の三つの方法から選びます。

1

文章をコピーして貼る

WebページやPDFで文字を選択できる場合

2

写真・スクリーンショットを添付する

紙のテキスト、図・表・擬似言語を含む問題の場合

3

必要な部分を手で入力する

コピーも画像添付もできない場合

方法1 問題文をコピーして貼り付ける

問題文、擬似言語、選択肢をまとめてコピーし、質問の前に貼り付けます。長い問題では、次の見出しを付けるとAIが内容を区別しやすくなります。

AIへ貼り付ける形	
問題文	ここに問題文を貼る
擬似言語	ここに擬似言語を貼る
選択肢	ここに選択肢を貼る
自分が分からないところ	例: 3行目で、totalにどの値が入るのか分からない

方法2 紙のテキストを撮影する／スクリーンショットを添付する

紙のテキストは、問題全体が読めるように写真を撮ってアップロードします。WebページやPDFでは、スクリーンショットを添付できます。どちらも問題文、擬似言語、選択肢が途中で切れないようにしてください。影や手で文字が隠れないようにし、複数ページなら順番に添付します。

画像を添付したら、最初に読み取り確認をする

AIは、画像内の添字、記号、数字を読み間違えることがあります。すぐに解かせず、「画像から読み取った問題文と擬似言語をそのまま書き出してください」と頼み、元画像と見比べます。

方法3 必要な部分を手で入力する

画像を添付できない場合は、問題の条件と擬似言語を入力します。すべてを打ち直すのが大変なら、分からない行と、その行で使う変数の初期値だけでも構いません。

AIへ送る順番

問題を渡す



読み取りを確認する



分からない点を書く



ヒントを頼む

0.5 自習での質問は「最初の設定」と「問題ごとの質問」に分ける

研修中は、Gemini Notebookの出力カスタマイズにより、答えや完成コードを見せない設定が済んでいます。そのため、0.3の手順で問題番号と質問内容を送るだけで構いません。

研修後に別の参考書で自習するときは、チャットを始めた直後に**最初の設定**を一度だけ送ります。同じチャットを続ける間は、毎回繰り返す必要はありません。新しいチャットを始めたときだけ、もう一度設定します。

手順1 新しいチャットの最初に、学習方法を設定する

AI

最初の設定として送る文

あなたはプログラミング未経験者の学習支援者です。問題について、正解や完成した擬似言語はすぐには示さず、ヒントや問いかけを通じて私が自力で考えられるように分かりやすくガイドしてください。私が考え方を説明したときは、どこに誤りがあるかや、次に確認すべきポイントを親切にアドバイスしてください。

手順2 問題ごとに、質問したいことを送る

問題を0.4の方法で渡した後、分かっていること・試したこと・分からない点を添えて質問します。自分で書いた解答やトレース表は、画像または文章で追加します。

AI

問題ごとに質問するとき

添付した問題について質問です。前の合計に次の数を足すとき、どの値を使うのか分かりません。最初の足し算までは紙に書きました。次にどこに着目すればよいか教えてください。

発展 同じ考え方で解ける「類題」を作ってもらう

問題を解き終えた後、理解を深めるためにAIへ類題を作ってもらうのも効果的です。

AI

AIへの質問例 | 類題を作るとき

指定した問題と同じ考え方で解ける類題を一問作ってください。AIが作成した問題であることを明記し、条件に矛盾がないかもチェックしてください。

0.6 AIの回答を確認する

AIから回答を得たら、次の順番で確認します。

1. 問題文に書かれていない条件を勝手に追加していないか
2. 配列の先頭の添字を取り違えていないか
3. 繰返しの開始値・終了値を取り違えていないか
4. 代入前の値と代入後の値を混同していないか

5. 自分でトレースして同じ結果になるか

個人情報・会社情報を入力しない

氏名、メールアドレス、顧客情報、公開されていない会社資料、パスワードなどはAIへ入力しません。この教材の練習問題のように、公開しても問題のない内容だけを使います。

第1章 アルゴリズムと擬似言語

1.1 アルゴリズムとは

アルゴリズムとは、目的を達成するための**処理手順**です。

身近なたとえ

アルゴリズムは「料理のレシピ」と同じ

カレーを作るとき、材料だけを渡されても完成しません。「切る → 炒める → 煮る」のように、作業の順番が必要です。コンピュータにも、同じように具体的な手順を一つずつ伝えます。



例えば、「三つの数の中から最大の数を見つける」という目的に対して、次の手順を考えられます。

1. 一つ目の数を、現在の最大値として覚える
2. 二つ目の数が現在の最大値より大きければ、最大値を更新する
3. 三つ目の数も同じように比較する
4. 最後に残った最大値を表示する

人間が何となく行っている判断を、コンピュータが実行できる順序に分解したものがアルゴリズムです。

例: 3、10、7の中から最大値を探す

3

最初の暫定1位

10

3より大きいので、暫定1位を10へ交代

7

10より小さいので、そのまま

最後の暫定1位「10」が答え

1.2 良いアルゴリズム

アルゴリズムには、少なくとも次の性質が必要です。

- 手順が明確である
- 同じ入力に対して、決められた結果になる
- いつか処理が終了する
- 実行できない曖昧な指示がない

「いい感じに並べる」「適切な回数だけ繰り返す」という表現は、人間には伝わってもコンピュータには伝わりません。

人には通じるかもしれない

「数字をいい感じに並べて」

何を基準に？ 大きい順？ 小さい順？

コンピュータにも伝わる

「左から右へ、小さい順に並べる」

基準と終了地点が明確

1.3 擬似言語とは

擬似言語は、アルゴリズムをプログラムに近い形で表現するための記述方法です。特定のプログラミング言語に依存せず、処理の考え方を表します。



本書では、IPAが定める基本情報技術者試験用の擬似言語の記述形式に合わせます。

はじめて出てくる記号と言葉の読み方

次の例には、これから何度も使う基本の書き方が含まれています。分からない英単語や記号があっても、最初は一つずつ意味を対応させれば大丈夫です。

書き方	読み方・役割
整数型: price	price という名前の「整数を入れる箱」を用意する。整数型は、小数点のない数(0、150、-3 など)を入れるという意味。
price ← 150	右側の 150 を、左側の箱 price に入れる。← は「代入(だいにゅう)」といい、等しいという意味ではない。
price × count	price の値と count の値を掛け算する。× は掛け算、+ は足し算、÷ は割り算。
表示する(total)	total に入っている値を画面に見せる。丸括弧の中は、表示する対象。

ここでは、price、count、total を変数と呼びます。変数とは、値を覚えておくための「名前付きの箱」です。英単語そのものを暗記する必要はありません。何を入れている箱かを考えて読んでください。

整数型: price

整数型: count

整数型: total

price ← 150

count ← 3

total ← price × count

表示する(total)

上から一行ずつ実行すると、total には450が入り、450が表示されます。

※ 画面への表示と値の入力

本書では、画面に値を表示する操作を 表示する(値)、外部から値を受け取る操作を 入力する() と表記します。実際の試験でも、問題文の指示に合わせて同様の表現が使われます。

1.4 処理を作る三つの基本構造

解説のフローチャートの読み方

フローチャートは、処理の進み方を図で表したものです。まず「開始」から矢印を一つずつたどります。

図の形	意味	読み方
丸い開始・終了	処理の入口と出口	開始から読み、終了で止める
四角い箱	実行する処理	箱の中を上から順に行う
ひし形	条件判定	「はい(真)」か「いいえ(偽)」の片方だけへ進む
矢印	次に進む先	上へ戻る矢印なら、戻った条件をもう一度確かめる

繰返しもひし形で「まだ続けるか」を確認します。詳しい読み方は第5章でも練習します。

複雑なアルゴリズムも、基本的には次の三つを組み合わせで作ります。

1

順次

上から順番に行う

例: 服を着てから靴を履く

2

選択

条件で行動を変える

例: 雨なら傘を持つ

3

繰返し

同じことを何度か行う

例: 全員分の出席を取る

順次(じゅんじ)

上から下へ、決められた順番で一つずつ処理を行います。

```
a ← 5
b ← 10
c ← a + b
```

1行ずつ順番に実行され、前の処理が終わってから次の処理へ進みます。

選択(せんたく)

条件によって、実行する処理を分けます。例えば「雨が降っていれば傘を持つ、降っていなければ傘を持たない」のように、条件の成立(はい／いいえ)に応じて進む道を変えます。

※ 擬似言語での詳しい書き方(`if` 文など)は、第4章でじっくり学びます。

繰り返し(くりかえし)

同じ処理を、決まった回数や条件を満たすまで繰り返します。例えば「名簿の全員分を1人ずつ確認する」のように、同じ作業を何度も行うときに使います。

※ 擬似言語での詳しい書き方(`for` 文や `while` 文)は、第5章でじっくり学びます。

理解チェック

「合計が100以上なら割引し、商品ごとに同じ計算を繰り返す」という処理には、三つの基本構造のうち何が含まれますか。

演習1-A 朝の手順

難易度 ★☆☆

「靴を履く→靴下を履く→外へ出る」をやり直しが起きない順序に直してください。また、このように上から決められた順序で一つずつ進む処理を、三つの基本構造で何と呼びますか。

解答欄

正しい順序

基本構造の名称

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Aについて質問です。手順の順番を入れ替えると結果が変わってしまう理由と、上から順に進む処理を「順次」と呼ぶ理由を教えてください。

演習1-B 雨の日の準備

難易度 ★☆☆

「出かける前に空を見て、雨なら傘を持ち、晴れなら帽子をかぶる」という処理があります。条件によって進む道を変える処理を、三つの基本構造で何と呼びますか。また、この例での条件は何ですか。

解答欄

基本構造の名称

この例での条件

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Bについて質問です。天候によって行動を分ける処理が「選択」構造と呼ばれる理由と、条件を「真」と「偽」に整理するコツを教えてください。

演習1-C 同じ作業の繰り返し

難易度 ★★☆☆

「10枚の書類すべてに確認印を押す」という作業があります。三つの基本構造のどれに当たりますか。また、この処理が終わるのはどんな条件のときですか。

解答欄

基本構造の名称

終了する条件

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Cについて質問です。同じ動作を繰り返すときに「いつ終わるか(終了条件)」をどう考えればよいか、繰り返し構造のポイントを教えてください。

演習1-D 曖昧な指示とアルゴリズム

難易度 ★★☆☆

コンピュータに「数字をいい感じに並べて」と指示しても正しく動かないのはなぜですか。アルゴリズムに必要な性質から理由を説明してください。

解答欄

正しく動かない理由

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Dについて質問です。人には通じる「いい感じに並べて」という指示が、なぜコンピュータには実行できないのか、アルゴリズムの観点から教えてください。

演習1-E 変数と代入の役割

難易度 ★★★

変数の役割と代入記号「←」の意味を、それぞれ説明してください。

解答欄

変数の役割

代入「←」の意味

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Eについて質問です。変数が値を保持する仕組みと、代入の「←」が数学の等号「=」と違って「右辺を左辺に入れる」という意味になる理由を教えてください。

演習1-F 三つの基本構造の組み合わせ

難易度 ★★☆☆

「提出された答案用紙を1枚ずつ確認し、60点以上なら合格印、60点未満なら再試印を押し、全員分終わるまで続ける」という作業があります。三つの基本構造(順次・選択・繰返し)のうち、どれが含まれていますか。また、どの作業がどの構造に対応しているか説明してください。

解答欄

含まれる基本構造

それぞれの対応関係

AI

AIへの質問例 | 考え方や疑問点を伝える

問題1-Fについて質問です。答案確認の手順の中で、「順次」「選択」「繰返し」の3つの基本構造がそれぞれどの作業に対応しているか教えてください。

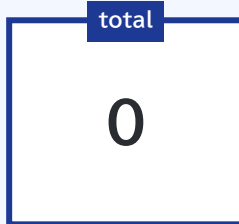
第2章 変数・データ型・代入

2.1 変数は値を覚える場所

プログラムでは、計算途中の値を覚えておく必要があります。その保管場所が**変数**です。

変数には名前を付けます。例えば、合計を保存する変数なら `total`、件数なら `count` のように、役割が分かる名前を使います。

変数は「名前付きの箱」



箱の名前: `total`

箱の中身: `0`

箱に入れられるもの: 整数

```
整数型: total  
total ← 0
```

この例では、整数を保存する変数 `total` を用意し、最初の値として0を入れています。

2.2 データ型

データ型は、変数にどのような値を保存するかを表します。「型」は値の種類という意味です。

データ型は、箱に貼る「中に何をに入れてよいか」というラベルです。整数用の箱に文章を入れないように、保存する値の種類をあらかじめ決めます。

身近なたとえ: 引っ越しの段ボール

引っ越しをするとき、段ボール箱にペンで「本」「服」「お皿」と記入しますよね。そうすると、その段ボールには該当する種類のものだけを入れることになります。

もし重い「本」と割れやすい「お皿」を一つの段ボールに混ぜてしまったらどうなるでしょうか。運ぶ途中で大切なお皿が割れてしまうかもしれませんし、後で荷解きして使うときにも、どこに何が入っているか探すのが大変になってしまいます。

コンピュータの変数もまったく同じです。箱(変数)には、種類(データ型)に合わせて決まった値だけを入れておくべきなのです。

データ型	保存する値の例	使用例
整数型	10、-3、0	件数、添字、合計
実数型	3.14、18.5	平均、身長、割合
文字型	'A'、'？'	一文字の記号
文字列型	"合格"、"Tokyo"	名前、メッセージ
論理型	true、false	条件が成立するか

実数型は、小数点を含めて扱える数を入れる型です。例えば身長 1.68 m、平均 72.5 点、消費税率 0.1 のような値に使います。論理型の true は「条件が成立する」、false は「条件が成立しない」という意味で、後の if や while の判定に使います。

数字と文字列は別物

整数の 10 は計算に使えますが、文字列の "10" は文字として扱われます。問題文で指定されたデータ型を確認しましょう。

2.3 代入は「右から計算して、左の箱へ入れる」

代入とは、計算結果や値を変数へ入れる操作です。矢印記号 ← を使って表します。

```
total ← total + 5
```

コンピュータは、この処理を次の順番で実行します。

1. まず右側を計算する:現在の `total` の値を取り出し、それに5を足す
2. 次に左側の箱へ入れる:計算した結果の値を、左側の `total` へ代入して上書きする

【重要】代入は「等しい」という意味ではありません！

一般的なプログラミング言語(PythonやC言語など)では代入を `=` と書きますが、数学の「等しい」ではなく「右辺の値を左辺へ入れる」という意味です。基本情報技術者試験の擬似言語では、誤解を防ぐために矢印記号 `←` を使って「右辺の計算結果を、左辺の箱へ代入する」と表します。決して「等しい」と読まないように注意しましょう。

例:実行前の `total` が3の場合

実行する文	右辺の計算	実行後の <code>total</code>
<code>total ← total + 5</code>	<code>3 + 5</code>	8

2.4 値は上書きされる

変数へ新しい値を代入すると、以前の値は失われます。

```
a ← 10  
a ← 3
```

実行後の `a` は3です。最初に入れた10は残りません。



新しい値を入れる →

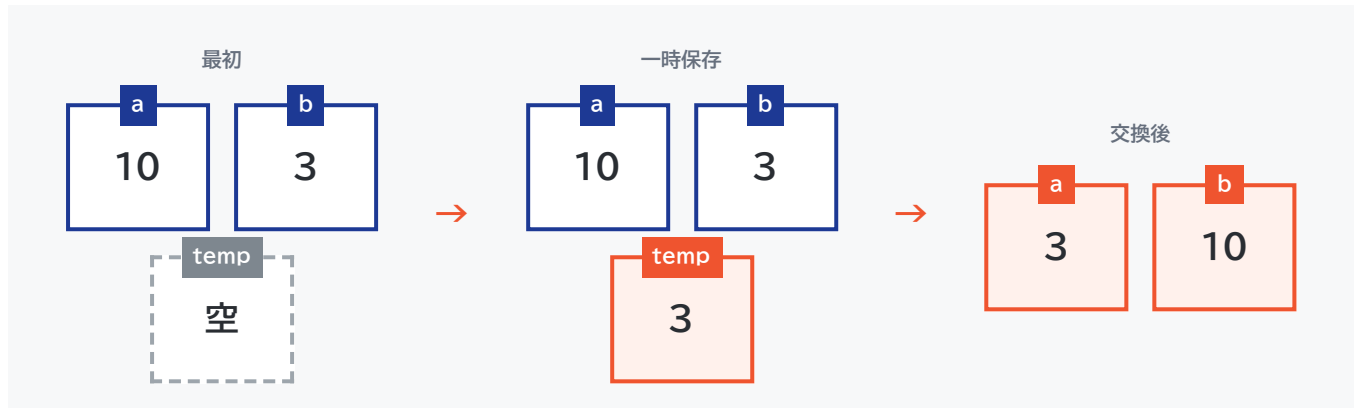


同じ箱の中身が10から3へ置き換わる

2.5 二つの変数の値を交換する

`a` に10、`b` に3が入っているとします。二つの値を交換するには、一時的に値を保存する変数が必要です。

二つのコップの飲み物を交換するとき、空のコップが一つ必要なのと同じです。`temp` は、値を一時的に避難させる空のコップです。



整数型: `temp`

```
temp ← b
b ← a
a ← temp
```

実行後	<code>a</code>	<code>b</code>	<code>temp</code>
初期状態	10	3	未定義
<code>temp ← b</code>	10	3	3
<code>b ← a</code>	10	10	3
<code>a ← temp</code>	3	10	3

よくある間違い

```
a ← b  
b ← a
```

一行目を実行した時点で、`a` に入っていた10が失われます。二行目では、どちらも3になってしまいます。

演習2-A 代入を追う

難易度 ★☆☆

次の処理が終わったとき、`total` の値はいくつですか。各行の実行後の値を教えてください。

整数型: `total`

```
total ← 2
```

```
total ← total + 3
```

```
total ← total × 2
```

解答欄

`total ← 2` 実行後

`total ← total + 3` 実行後

`total ← total × 2` 実行後(最終値)

AI

AIへの質問例 | 値の変化を追えないとき

問題2-Aについて質問です。右辺の`total`には、どの時点の値を使えばよいのでしょうか。私は各行の実行後の値を〇〇と考えたのですが、この考え方で合っていますか？

演習2-B 値を交換する

難易度 ★☆☆

x に7、y に12が入っています。実行後に x が12、y が7となるように、空欄①～③を埋めてください。

整数型: temp

temp ← [①]

x ← [②]

y ← [③]

解答欄

空欄 ①

空欄 ②

空欄 ③

AI

AIへの質問例 | 値の交換が分からないとき

問題2-Bについて質問です。一時変数tempが必要な理由と、空欄①～③の入れ替え順序が分かりません。考え方を教えてください。

演習2-C 同じ名前を何度も更新する

難易度 ★★☆☆

最後のaとbを答え、各行の実行後の値を表にしてください。

この問題の変数は整数型です。

```
a ← 4
b ← a
a ← a + 6
b ← b + a
```

実行する処理	実行後の a	実行後の b
a ← 4		
b ← a		
a ← a + 6		
b ← b + a		

解答欄

最後の a

最後の b

AI

AIへの質問例 | 考え方や疑問点を伝える

問題2-Cについて質問です。bへaを代入したあとでaの値を変更しても、bの値は勝手に変わらないのでしょうか？変数が値を保持する仕組みを教えてください。

演習2-D 小数を含む計算

難易度 ★★☆☆

price、rate、discountは実数型です。割引額discountと割引後のpriceを教えてください。rate=0.2は20%を表します。

実数型: price, rate, discount

```
price ← 1250
rate ← 0.2
discount ← price × rate
price ← price - discount
```

解答欄

割引額 discount

割引後 price

AI

AIへの質問例 | 考え方や疑問点を伝える

問題2-Dについて質問です。実数型のかけ算で割引額を求めてから元の金額から引く計算について、私は〇〇と考えたのですが、計算手順は合っていますか？

演習2-E 口座間の資金移動

難易度 ★★★

手数料なしで口座aから口座bへamount円移します。aの残高は移動額以上とします。処理後のa、b、totalの値を教えてください。

この問題の変数は整数型です。

```
a ← 5000
b ← 2000
amount ← 1200
a ← a - amount
b ← b + amount
total ← a + b
```

解答欄

a の値

b の値

total の値

AI

AIへの質問例 | 考え方や疑問点を伝える

問題2-Eについて質問です。口座aから口座bへ資金を移動する処理で、私は移動後の残高を a=〇〇、b=〇〇、total=〇〇 と計算したのですが合っていますか？

演習2-F 担当番号を一つずらす

難易度 ★★★

窓口a、b、cの担当番号を、aには元のb、bには元のc、cには元のaが入るよう更新します。処理後のa、b、cの値を教えてください。

この問題の変数は整数型です。

```
a ← 10
b ← 20
c ← 30
temp ← a
a ← b
b ← c
c ← temp
```

解答欄

a の値

b の値

c の値

AI

AIへの質問例 | 考え方や疑問点を伝える

問題2-Fについて質問です。3つの変数の値を順番に入れ替える際、一時変数tempを使って値が上書きされないようにする手順を教えてください。

第3章 順次処理

3.1 上から順番に実行する

順次処理は、文を上から下へ一度ずつ実行する、最も基本的な構造です。

① リンゴの単価を150円として覚える



② みかんの単価を80円として覚える



③ それぞれの代金を計算して足す



④ 合計金額を表示する

整数型: apple

整数型: orange

整数型: total

apple ← 150

orange ← 80

total ← apple × 3 + orange × 4

表示する(total)

この処理は、150円のリンゴを3個、80円のみかんを4個買ったときの合計金額を求めています。

計算する順番: 一つの式の中では、丸括弧の内側を先に計算し、次に掛け算・割り算、最後に足し算・引き算を行います。上の式では150×3と80×4を先に求め、その結果を足します。なお、複数の代入文は上から一行ずつ実行します。

3.2 入力・処理・出力で整理する

問題文を読んだら、最初に次の三つへ分けます。



分類	意味	例
入力	最初に与えられる値	単価、個数
処理	入力を使って行う計算	単価×個数、合計
出力	最後に求めるもの	合計金額

演習3-A 代金を計算する

難易度 ★☆☆

一つ150円のリンゴを3個、一つ80円のみかんを4個買います。合計金額を計算して表示するように、空欄を埋めてください。

```
整数型: apple
整数型: orange
整数型: total

apple ← [ ① ]
orange ← [ ② ]
total ← [ ③ ]
表示する(total)
```

解答欄・各変数の役割	
空欄 ①	-----
空欄 ②	-----
空欄 ③	-----
各変数の役割(自分の言葉で)	apple, orange, total の役割を一言で説明 -----

AI

AIへの質問例 | 問題文を整理するとき

問題3-Aについて質問です。商品の単価と個数から代金を計算する流れを整理しました。入力・処理・出力の分け方と計算式はこれで合っていますか？

演習3-B 消費税込みの金額

難易度 ★★★

商品の税抜価格 `price` と税率 `rate` が与えられています。税込価格を求めて表示する処理を作成してください。小数点以下の扱いは考えず、実数型で計算します。

実数型: `price`
実数型: `rate`
実数型: `taxIncluded`

`price` ← 1200
`rate` ← 0.1

// ここに処理を書く

解答欄

税込価格の計算式

表示する処理

AI

AIへの質問例 | 式を点検するとき

問題3-Bについて質問です。私が作った式は〇〇です。これは「消費税額そのもの」を求めているのか「税込価格」を求めているのか、どちらと判断すべきでしょうか？

演習3-C 入力から出力まで

難易度 ★☆☆

priceは入力された単価、quantityは入力された個数とします。単価200円、個数3個のとき何が表示されますか。入力・処理・出力に分けて説明してください。

この問題の変数は整数型です。

```
price ← 200
quantity ← 3
total ← price × quantity
表示する(total)
```

解答欄

表示される値

入力 price(単価)

入力 quantity(個数)

処理

出力

AI

AIへの質問例 | 考え方や疑問点を伝える

問題3-Cについて質問です。このプログラムで「入力」「処理」「出力」に当たる部分はそれぞれどれでしょうか？単価と個数は入力と考えてよいですか？

演習3-D 計算する順番

難易度 ★★☆☆

最後のpriceを求めてください。「割引後に送料を足す」と「送料を足してから割引く」が同じにならない理由も説明してください。金額は実数型で、小数点以下の丸めは行いません。

実数型: price

```
price ← 2000
price ← price × 0.9
price ← price + 300
```

解答欄

最後の price

同じにならない理由

AI

AIへの質問例 | 考え方や疑問点を伝える

問題3-Dについて質問です。「割引後に送料を足す」場合と「送料を足してから割引く」場合で結果が食い違うのは、計算の順序で何が変わってしまうからですか？

演習3-E 作業時間の見積り

難易度 ★★★

【実践】ある工場で製品を18個製造します。作業開始前の機械の準備・点検(全体の準備時間)として12分かかり、その後製品1個あたり7分の加工時間がかかります。全体の作業にかかる総所要時間 minutes(分)と、それを時間に換算した hours(時間)を正しく計算するように、プログラム中の空欄 [a], [b] に入る適切な組合せを一つ選んでください。hoursは実数型で、小数点以下を切り捨てません。

整数型: quantity, perItem, setup, minutes 実数型: hours

```
quantity ← 18
perItem ← 7
setup ← 12
minutes ← [ a ]
hours ← [ b ]
```

選択肢	内容
ア	a: quantity × perItem / b: minutes ÷ 60
イ	a: quantity × perItem + setup / b: minutes ÷ 60
ウ	a: (quantity + setup) × perItem / b: minutes ÷ 60
エ	a: quantity × perItem + setup / b: minutes × 60

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題3-Eについて質問です。準備時間と加工時間を、どのように分けて考えればよいか分かりません。

演習3-F 倉庫の一日の記録

難易度 ★★★

【実践】ある倉庫で商品の受払(在庫管理)を行います。朝の業務開始時点の在庫は40個(stock)です。日中に15個が入荷(received)し、その後23個を出荷(shipped)しました。一日の業務終了後、翌日の目標在庫数50個(target)に対して何個不足しているか(shortage)を計算します。在庫数の増減と不足数を正しく計算するように、プログラム中の空欄 [a]、[b] に入る適切な組合せを一つ選んでください。※出荷できる十分な在庫があるものとします。

この問題の変数は整数型です。

```
stock ← 40
received ← 15
shipped ← 23
stock ← [ a ]
stock ← [ b ]
shortage ← 50 - stock
```

選択肢	内容
ア	a: stock + received / b: shipped - stock
イ	a: received / b: stock - shipped
ウ	a: stock - received / b: stock + shipped
エ	a: stock + received / b: stock - shipped

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題3-Fについて質問です。入荷と出荷のたびに在庫がどう変わるか、整理の仕方が分かりません。

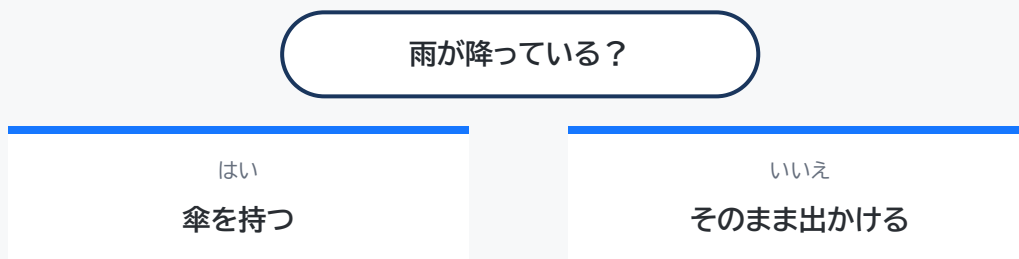
第4章 選択処理

4.1 条件によって処理を分ける

プログラムは通常、上から下へと順番に実行されます(順次処理)。しかし、実際の業務では「テストが60点以上なら合格、未満なら不合格」「在庫があれば出荷、なければ発注」のように、状況に応じて行う処理を切り替えた場面がたくさんあります。

このように、ある条件が成立するかどうかによって実行する処理を分岐させる仕組みを、アルゴリズムの世界では「**選択処理**」(または条件分岐)と呼びます。

選択は「雨が降っている？」と考えるのと同じ



擬似言語では、英語の `if` (もし～なら)、`else` (そうでなければ)、`endif` (ifの終わり)を使って次のように記述します。

```
if (score ≥ 60)
    表示する("合格")
else
    表示する("不合格")
endif
```

`if` 文の基本構文と読み方

コードを上から順に追うと、次の4つのステップで処理が進みます。

1. **`if` (条件式)**: カッコ内の条件が成り立っているか(真か偽か)を判定します。ここでは「`score` の値が60以上か？」を調べます。
2. **条件成立(真)の処理**: 条件が成り立っていれば、`if` の直後(`else` の手前まで)を実行します。この例では「表示する("合格")」が行われます。実行後は、`else` の処理を飛ばして `endif` の先へ進みます。

3. **else** (条件不成立・偽の処理): 条件が成り立っていなければ、**if** の直後をスキップして、**else** の直後を実行します。この例では **表示する("不合格")** が行われます。
4. **endif**: 「ここで条件分岐の範囲が終わる」という目印です。分岐した処理はここで合流し、次の命令へ進みます。

インデント(字下げ)の役割

if や **else** の内側にある処理は、半角スペース等で字下げ(インデント)して記述します。「どの行が分岐の内側の処理なのか」を目で見て分かりやすくするための大切な書き方です。

4.2 比較演算子

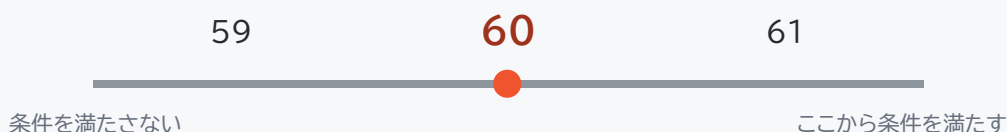
演算子	意味	例
=	等しい	$x = 10$
≠	等しくない	$x \neq 10$
>	より大きい	$x > 10$
<	より小さい	$x < 10$
≧	以上	$x \geq 10$
≦	以下	$x \leq 10$

※ 試験で使う関係演算子

キーボード入力によく使う $>=$ や $<=$ ではなく、試験問題では $\geq$ や $\leq$ を使います。代入は $\leftarrow$ 、等しいかの比較は $=$ です。

「18より大きい」と「18以上」は異なります。境界となる値を確認しましょう。

「60点以上」の境界を見る



一般的な演算子(% や ++ / --)と擬似言語の表現

条件判定やループのカウント処理などでよく使われる演算について、一般的なプログラミング言語(Python、C言語、Javaなど)の記号と、試験の擬似言語での書き方を整理しておきましょう。

演算の種類	一般的な言語の記号	擬似言語での書き方	意味と具体例
割り算の余り	% (例: <code>x % 2</code>)	mod (例: <code>x mod 2</code>)	割った余りを求めます。 <code>7 mod 3</code> は <code>1</code> ($7 \div 3 = 2$ 余り 1)。 <code>x mod 2 = 0</code> であれば「2で割り切れる(偶数)」と判定できます。
1増やす (インクリメント)	++ (例: <code>x++</code>)	<code>x ← x + 1</code>	変数の値を1増やして上書き代入します。 擬似言語には ++ のような省略記法はありません。
1減らす (デクリメント)	-- (例: <code>x--</code>)	<code>x ← x - 1</code>	変数の値を1減らして上書き代入します。 擬似言語では引き算の式をそのまま書きます。

※ 擬似言語には省略演算子がない

一般的なプログラミング言語には `i++` や `total += x` のような省略記法がありますが、基本情報技術者試験の擬似言語では必ず `i ← i + 1` や `total ← total + x` のように、「右辺で計算して左辺の変数へ代入する」素直な形式で記述します。

4.3 複数の条件

二つ以上の条件を組み合わせるときは、論理演算を使います。

and

学生証も必要
予約票も必要

両方そろって通れる

or

学生証または
運転免許証

どちらか一方で通れる

not

「雨ではない」

条件を反対にする

論理演算	成立する条件
and	両方の条件が成立
or	少なくとも一方が成立
not	条件が成立しない

```
if (age ≥ 18 and age < 65)
    表示する("対象")
endif
```

この例のように、`else` は省略できます。条件が偽なら内側の処理は行わず、`endif` の後へ進みます。

4.4 三つ以上に分ける

`elseif` (エルスイフ)は「それまでの条件が成立しなかった場合に、別の条件を調べる」という意味です。`if` で最初の条件を調べ、成立しなければ `elseif` へ進みます。`else` は、どの条件にも当てはまらなかった場合の処理です。`elseif` と違い、`else` には条件を書きません。

次の例では、まず80点以上かを調べます。そうでなければ60点以上かを調べ、どちらでもなければCを表示します。

```
if (score ≥ 80)
    表示する("A")
elseif (score ≥ 60)
    表示する("B")
else
    表示する("C")
endif
```

上から条件を確認し、最初に成立した処理だけを実行します。そのため、条件を書く順番が重要です。

80点以上?	A
成立しなければ次へ ↓	
60点以上?	B
成立しなければ次へ ↓	
どちらでもない	C

順番を逆にするとうなるか

```
if (score ≥ 60)
  表示する("B")
elseif (score ≥ 80)
  表示する("A")
endif
```

score が90でも、最初の `score ≥ 60` が成立するため「B」と表示されます。

演習4-A 偶数・奇数を判定する

難易度 ★☆☆

整数 x が偶数なら「偶数」、そうでなければ「奇数」と表示します。空欄を埋めてください。

※ヒント: x を2で割った余りが、どうなれば偶数で、どうなれば奇数でしょうか？

```
if ([ ① ])
    表示する("偶数")
else
    表示する("奇数")
endif
```

解答欄

空欄 ①(条件式)

AI

AIへの質問例 | 偶数・奇数の判定

問題4-Aについて質問です。偶数と奇数は、2で割った余りを使ってどのように見分けますか？

演習4-B 合格の境目

難易度 ★☆☆

scoreに59、60をそれぞれ入れたときの表示を教えてください。 $\geq$ は「以上」を表します。

```
整数型: score
文字列型: result

if (score  $\geq$  60)
    result  $\leftarrow$  "合格"
else
    result  $\leftarrow$  "再挑戦"
endif
表示する(result)
```

解答欄

score=59 の表示

score=60 の表示

AI

AIへの質問例 | 境界値の判定理由

問題4-Bについて質問です。条件式「score $\geq$ 60」のとき、ちょうど60点の場合は合格と再挑戦のどちらになるのでしょうか？境界値の判定理由を教えてください。

演習4-C BMIを判定する

難易度 ★★☆☆

身長height(m)と体重weight(kg)からBMIを求め、次の基準で結果を表示します。(計算式: BMI = weight ÷ (height × height) / 基準: 18.5未満=低体重、18.5以上25未満=標準体重、25以上=肥満)

実数型: height, weight, bmi 文字列型: result

```
height ← 入力する()
weight ← 入力する()
bmi ← weight ÷ (height × height)

if ([ ① ])
  result ← "低体重"
elseif ([ ② ])
  result ← "標準体重"
else
  result ← "肥満"
endif
表示する(result)
```

解答欄

空欄 ①(条件式)

空欄 ②(条件式)

BMI 18.49 / 18.50
の期待結果

BMI 24.99 / 25.00
の期待結果

AI

AIへの質問例 | 考え方や疑問点を伝える

問題4-Cについて質問です。私が考えた条件式は ①が〇〇、②が〇〇です。18.5や25.0ちょうどの場合に、判定漏れや重複が起きないか教えてください。

演習4-D 二つの条件を満たす

難易度 ★★☆☆

ageは年齢、hasTicketは券ありなら1、なしなら0です。入場には12歳以上かつ券ありが必要です。空欄①に入る条件式を教えてください。また、(age, hasTicket)が(12,1)、(11,1)、(20,0)のときの結果も教えてください。

整数型: age, hasTicket 文字列型: result

```
if ([ ① ])
    result ← "入場可"
else
    result ← "入場不可"
endif
```

解答欄

空欄 ①(条件式)

(12, 1) の結果

(11, 1) の結果

(20, 0) の結果

AI

AIへの質問例 | 考え方や疑問点を伝える

問題4-Dについて質問です。12歳以上と券ありの両方を条件にするには、二つの条件をどのようにつなげればよいですか？

演習4-E 利用料金を判定する

難易度 ★★★

【実践】年齢ageは0以上の整数です。6歳未満は無料(0円)、6歳以上18歳未満は400円、18歳以上は900円です。年齢に応じた料金feeを正しく判定するように、プログラム中の空欄 [a]、[b] に入る適切な組合せを一つ選んでください。

この問題の変数は整数型です。

```
if ([ a ])
  fee ← 0
elseif ([ b ])
  fee ← 400
else
  fee ← 900
endif
```

選択肢	内容
ア	a: age < 6 / b: age < 18
イ	a: age ≤ 6 / b: age < 18
ウ	a: age < 6 / b: age ≤ 18
エ	a: age ≤ 6 / b: age ≤ 18

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題4-Eについて質問です。「6歳未満」や「18歳未満」を判定するとき、条件式の記号は「<」と「≤」のどちらを使うべきか理由とともに教えてください。

演習4-F 配送サービスを選ぶ

難易度 ★★★

【実践】冷蔵が必要(cold=1)なら金額によらず冷蔵便600円、不要(cold=0)なら購入額amountが5000円以上で送料無料(0円)、未満で通常便300円です。送料feeを正しく判定するように、プログラム中の空欄 [a]、[b] に入る適切な組合せを一つ選んでください。

この問題の変数は整数型です。

```
if ([ a ])
  fee ← 600
elseif ([ b ])
  fee ← 0
else
  fee ← 300
endif
```

選択肢	内容
ア	a: amount $\geq$ 5000 / b: cold = 1
イ	a: cold = 0 / b: amount $\geq$ 5000
ウ	a: cold = 1 / b: amount $\geq$ 5000
エ	a: cold = 1 / b: amount < 5000

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題4-Fについて質問です。冷蔵便の判定と通常便の送料無料判定は、どちらを先に書かないと誤判定になってしまうのでしょうか？

第5章 繰り返し処理

5.1 同じ処理を繰り返す

プログラムで同じ処理を何十回、何百回と行いたいとき、同じ命令を何行もコピーして書くのは大変ですし、修正の手間や間違いの原因になります。

そこで使われるのが「**繰り返し処理**」(ループ処理)です。繰り返し処理を使えば、「指定した回数」や「ある条件を満たしている間」、同じ処理をコンピュータに自動で繰り返し実行させることができます。

5人の出席を確認する



「名前を呼ぶ → 返事を記録する」を5回繰り返す

擬似言語には、主に `while` と `for` という二つの繰り返し構文があります。本研修では、繰り返しの仕組みが見えやすい `while` を先に学び、その書き方を簡潔にした `for` へ進みます。

5.2 まずwhileで繰り返しの仕組みを知る

`while` (ホワイル)は「条件が成立している間」という意味です。次の例は、`i` を1から5まで変えながら表示します。

```
整数型: i

i ← 1
while (i ≤ 5)
  表示する(i)
  i ← i + 1
endwhile
```

このように `i` で回数を数える `while` には、次の三つが必要です。

必要な部分	この例	役割
① 初期化	<code>i ← 1</code>	数え始める値を決める
② 継続条件	<code>while (i ≤ 5)</code>	条件が真の間だけ処理を続ける
③ 更新	<code>i ← i + 1</code>	次の回へ進み、いつか条件を偽にする

`endwhile` は、繰り返す範囲の終わりです。内側の処理は字下げ(インデント)して、どこまでが繰り返しの範囲なのかを読みやすくします。

while を読む順番

1. 繰り返しの前に `i` を初期化する。
2. `while` の条件を判定する。
3. 条件が真なら、内側の処理を実行する。
4. `i ← i + 1` で値を更新する。
5. `while` の条件判定へ戻る。条件が偽なら終了する。



更新を忘れると終わらない

この例で `i ← i + 1` を忘れると、`i` は1のままです。`i ≤ 5` がいつまでも真になり、処理が終わりません。`i` で回数を管理するwhileでは、初期化・条件・更新の三点を必ず確認します。

なお、すべての `while` が `i` を使うわけではありません。「待っている人がいる間」のように別の状態を条件にする場合もあります。どの場合も、繰り返しの中で条件がいつか偽になる仕組みが必要です。

5.3 whileを簡潔に書くfor

1から5までのように回数が最初から決まっている場合、`while` では初期化・条件・更新を毎回別々に書くため、少し手間がかかります。`for` (フォー)は、この三つを先頭の一行にまとめて書けます。

同じ処理をwhileとforで比べる

コード中の `//` から右側は、人が読むための説明(コメント)です。計算として実行する命令ではありません。

`while` では、三つの要素を別々の場所に入ります。

```
i ← 1           // 初期化
while (i ≤ 5)    // 継続条件
  表示する(i)
  i ← i + 1      // 更新
endwhile
```

`for` では、同じ内容を一行にまとめます。

```
for (i を 1 から 5 まで 1 ずつ増やす)
  表示する(i)
endfor
```

whileで書く部分	forの一行に含まれる表現
i ← 1	i を 1 から
i ≤ 5	5 まで
i ← i + 1	1 ずつ増やす

つまり、`for` で初期化や更新が不要になったのではなく、`for`の先頭行にまとめて指定されているのです。`endfor` までの処理が終わるたびに、`i` は自動的に1ずつ増えます。

`endfor` は繰返す範囲の終わりです。更新後の値が指定範囲を超えたら、繰返しを終えて次の行へ進みます。例えば1から5までなら、5の回の後に6へ更新され、6の回は実行しません。開始値が最初から範囲を超えていれば、処理本体は一度も実行しません。

「1から5まで」は5を含む

`for (i を 1 から 5 まで 1 ずつ増やす)` では、`i` は1、2、3、4、5と変化し、処理本体は5回実行されます。

使い分けの目安

while:「待っている人がいる間」のように、条件を満たしている間だけ続けるとき

for:「名簿の10人を順に確認する」のように、回数や範囲が最初から分かっているとき

5.4 合計を求める

1から5までの合計を求めます。

ここで最も大切なのは、合計を覚える変数を繰返しの前に用意し、初期値0を入れておくことです。この例では、整数型: `total` が変数の宣言、`total ← 0` が初期化です。

```
整数型: total    // 合計用の変数を宣言する
total ← 0       // 繰返しの前に一度だけ0を入れる
```

合計用の変数を繰返しの中で初期化しない

`total ← 0` を繰返しの中に書くと、1周するたびに合計が0へ戻ります。前の周までに足した値が消えるため、合計を蓄積できません。変数名を `sum` にした場合も同じで、`sum ← 0` は繰返しの前に書きます。

次の書き方は誤りです。

```
for (i を 1 から 5 まで 1 ずつ増やす)
  total ← 0           // 毎回0に戻ってしまう
  total ← total + i
endfor
```

total は、数字を貯める貯金箱



整数型: i

整数型: total

```
total ← 0
```

```
for (i を 1 から 5 まで 1 ずつ増やす)
```

```
    total ← total + i
```

```
endfor
```

表示する(total)

i	実行前の total	total + i	実行後の total
1	0	1	1
2	1	3	3
3	3	6	6
4	6	10	10
5	10	15	15

累積する変数

`total` のように、繰返しのたびに値を加えていく変数を累積用の変数として使います。合計の初期値は通常0です。

5.5 件数を数える

1から10までの整数のうち、偶数の件数を数えます。

`count` は、条件に合ったときだけ押すカウンター



色の付いた偶数だけで、カウンターを1増やす

整数型: i

整数型: count

```
count ← 0
```

```
for (i を 1 から 10 まで 1 ずつ増やす)
```

```
  if (i mod 2 = 0)
```

```
    count ← count + 1
```

```
  endif
```

```
endfor
```

表示する(count)

5.6 合計から平均を求める

平均は「合計 ÷ 件数」で求めます。例えば2、4、9の平均は、合計15を3個で割った5です。割り切れない場合は小数になるため、平均を覚える変数には実数型を使います。件数が0のときは0で割れないので、平均を計算できません。

5.7 繰返しの中で、もう一度繰り返す

繰返しの内側に別の繰返しを書くことを、二重ループ(繰返しの入れ子)といいます。例えば、2段の棚に各3個の箱がある場合、「段を選ぶ」外側の繰返しの中で、「その段の箱を順に確認する」内側の繰返しを行います。

整数型: `i, j`

```
for (i を 1 から 2 まで 1 ずつ増やす)
  for (j を 1 から 3 まで 1 ずつ増やす)
    表示する(i)
    表示する(j)
  endfor
endfor
```

外側の <i>i</i> (段)	内側の <i>j</i> (箱の位置)が進む順番	箱を確認する回数
1	1 → 2 → 3	3回
2	1 → 2 → 3	3回

外側が1回進む間に、内側は最初から最後まで一巡します。内側が終わるまで `i` は変わりません。次の段に進むと、内側の `for` を再び始めるので、`j` は開始値1からやり直します。全体では $2 \times 3 = 6$ 回、箱を確認します。

文字を行ごとに表示する場合も同じです。内側で1行分の文字を表示し、内側の `endfor` の後で改行すれば、1行を表示するたびに一度だけ改行できます。

演習5-A 合計と平均

難易度 ★★☆☆

1から10までの整数を順番に表示し、最後に合計と平均を表示します。空欄を埋めてください。

```
整数型: i
整数型: total
実数型: average

total ← [ ① ]

for (i を [ ② ] から [ ③ ] まで 1 ずつ増やす)
  表示する(i)
  total ← [ ④ ]
endfor

average ← [ ⑤ ]
表示する(total)
表示する(average)
```

解答欄

空欄 ①

空欄 ②

空欄 ③

空欄 ④

空欄 ⑤

AI

AIへの質問例 | 繰返しを追うとき

問題5-Aについて質問です。`i`、実行前の `total`、実行後の `total` を記録するトレース表を埋めてみました。最初の2周分の考え方は合っていますか？

演習5-B 処理回数を数える

難易度 ★★☆☆

次の処理で、処理①と処理②はそれぞれ何回実行されますか。

```
整数型: i

i ← 1

while (i ≤ 5)
  // 処理①
  if (i mod 2 = 1)
    // 処理②
  endif
  i ← i + 1
endwhile
```

i	i ≤ 5	処理①	i mod 2 = 1	処理②
1				
2				
3				
4				
5				
6				

解答欄

処理①の実行回数	
処理②の実行回数	
終了時の i の値	

AI

AIへの質問例 | 自分の表を点検するとき

問題5-Bについて質問です。最後に条件が偽になるときも、判定回数に含めるのでしょうか。私が作成した表を確認し、条件判定と処理実行を混同している箇所がないか教えてください。

演習5-C 三回足す

難易度 ★☆☆

最後のtotalを求め、繰返し部分が何回実行されるか教えてください。

この問題の変数は整数型です。

```
total ← 0
for (i を 1 から 3 まで 1 ずつ増やす)
  total ← total + 2
endfor
```

解答欄

最後の total

繰返しの実行回数

AI

AIへの質問例 | 考え方や疑問点を伝える

問題5-Cについて質問です。for文のループで毎回2を加算するとき、totalの値が0からどのように増えていくか、変化の追い方を教えてください。

演習5-D 最初の判定が偽なら

難易度 ★☆☆

表示される*i*はいくつですか。繰返し内部は何回実行されますか。

この問題の変数は整数型です。

```
i ← 5
while (i < 5)
  i ← i + 1
endwhile
表示する(i)
```

解答欄

表示される *i*

内部の実行回数

AI

AIへの質問例 | 考え方や疑問点を伝える

問題5-Dについて質問です。while文はループの最初で条件判定を行いますが、最初から条件が偽だった場合は一度も中を実行しないのでしょうか？

演習5-E 目標金額まで積み立てる

難易度 ★★★

【実践】最初の貯金は1000円で毎月500円を積み立て、3000円以上になったら終了します。目標達成まで正しく繰り返すように、プログラム中の空欄 [a]、[b] に入る適切な組合せを一つ選んでください。

この問題の変数は整数型です。

```
balance ← 1000
months ← 0
while ([ a ])
    balance ← balance + 500
    months ← [ b ]
endwhile
```

選択肢	内容
ア	a: balance ≤ 3000 / b: months + 1
イ	a: balance < 3000 / b: months + 1
ウ	a: balance > 3000 / b: months + 1
エ	a: balance < 3000 / b: balance + 1

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題5-Eについて質問です。「終了する条件」と「繰り返す条件」を、どう区別すればよいですか？

演習5-F 一括処理の所要時間

難易度 ★★★

【実践】1番から6番までの伝票を順に処理します。各伝票に2分、3番と6番(3の倍数)には追加点検で各5分かかります。合計時間minutesを正しく計算するように、プログラム中の空欄 [a], [b] に入る適切な組合せを一つ選んでください。

この問題の変数は整数型です。

```
minutes ← 0
for (i を 1 から 6 まで 1 ずつ増やす)
  minutes ← [ a ]
  if ([ b ])
    minutes ← minutes + 5
  endif
endfor
```

選択肢	内容
ア	a: minutes + 2 / b: $i \bmod 2 = 0$
イ	a: 2 / b: $i \bmod 3 = 0$
ウ	a: minutes + i / b: $i \bmod 3 = 0$
エ	a: minutes + 2 / b: $i \bmod 3 = 0$

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題5-Fについて質問です。全伝票の処理時間を足しつつ、「3の倍数の伝票だけ追加時間を足す」には、どのような条件式を書けばよいでしょうか？

第6章 配列

6.1 配列は同じ型の値をまとめる

配列は、同じデータ型の複数の値を、一つの名前で管理する仕組みです。例えば、10人分の点数を一つずつ別の変数に入れる代わりに、`A` という一つの配列にまとめます。

配列は「番号付きのロッカー」

A[1]	A[2]	A[3]
5	10	3

配列名は建物の名前、添字はロッカー番号、値は中に入っている荷物です。

整数型の配列: `A ← {5, 10, 3}`

この例では、配列の要素番号を1から始めます。

要素	A[1]	A[2]	A[3]
値	5	10	3

`A` は配列全体の名前、`A[2]` は二番目の要素を表します。角括弧の中の番号を**添字(そえじ)**といい、どの箱を使うかを指定する番号です。`A[2]` は「Aの2番の箱」と読んでください。

※ 配列の要素番号を確認する

試験問題では、「配列の要素番号は1から始まる」のように開始位置が示されます。`A[1]` を先頭と決めつけず、問題文の指定を確認します。

6.2 配列の要素を順番に表示する

整数型: i

整数型の配列: $A \leftarrow \{3, 8, 2, 5\}$

for (i を 1 から 4 まで 1 ずつ増やす)

表示する($A[i]$)

endfor

i が 1、2、3、4 と変化するため、 $A[1]$ 、 $A[2]$ 、 $A[3]$ 、 $A[4]$ を順番に参照します。

$A[1]$	$A[2]$	$A[3]$	$A[4]$
3	8	2	5

↑ $i = 2$ なら、2 番のロッカー $A[2]$ を見る

6.3 配列の合計

整数型: i

整数型: $total$

整数型の配列: $A \leftarrow \{3, 8, 2, 5\}$

$total \leftarrow 0$

for (i を 1 から 4 まで 1 ずつ増やす)

$total \leftarrow total + A[i]$

endfor

表示する($total$)

i	A[i]	実行前の total	実行後の total
1	3	0	3
2	8	3	11
3	2	11	13
4	5	13	18

$$0 + A[1] \text{ の } 3 \quad 3 + A[2] \text{ の } 8 \quad 11 + A[3] \text{ の } 2 \quad 13 + A[4] \text{ の } 5 \quad 18$$

6.4 配列の最大値

整数型: i

整数型: max

整数型の配列: $A \leftarrow \{2, 5, 1, 9, 8, 10, 7, 3, 6, 4\}$

$\text{max} \leftarrow A[1]$

for (i を 2 から 10 まで 1 ずつ増やす)

if ($A[i] > \text{max}$)

max $\leftarrow A[i]$

endif

endfor

表示する(max)

最初の要素 $A[1]$ を、現在の最大値として max へ入れます。その後、残りの要素を一つずつ比較します。

max は「暫定チャンピオン」



今のチャンピオンより大きい値が現れたときだけ、 max を交代する

なぜ最大値の初期値を0にしないのか

配列の値がすべて負の数の場合、0は配列に存在しないのに最大値として残ってしまいます。最初の要素を初期値にすれば、配列内の値から最大値を選べます。

6.5 配列を先頭から探す

配列を先頭から一つずつ調べ、目的の値を探す方法を**線形探索(せんけいたんさく)**といいます。例えばA={6,2,9}から2を探すなら、A[1]の6と比べ、次にA[2]の2を見て一致を確認します。「探した値」は2、「見つかった位置」もこの例では2ですが、別のものです。9を探した場合は値が9、位置は3です。

見つかった位置を変数に記録する場合、添字が1から始まる配列では、位置として使わない0を「まだ見つからない」という目印にできます。同じ値が何度も現れる場合は、最初の位置と最後の位置のどちらを求めるかを確認します。

演習6-A 10個の合計

難易度 ★★☆☆

配列Aには、添字1から10までの10個の整数が入っています。合計を求めるように空欄を埋めてください。

```
整数型: i
整数型: total
整数型の配列: A

total ← [ ① ]

for (i を [ ② ] から [ ③ ] まで 1 ずつ増やす)
    total ← [ ④ ]
endfor

表示する(total)
```

解答欄	
空欄 ①	_____
空欄 ②	_____
空欄 ③	_____
空欄 ④	_____

AI

AIへの質問例 | 添字と値を整理するとき

問題6-Aについて質問です。iとA[i]を混同してしまいます。i、A[i]、totalがそれぞれ何を表しているか、私の理解を確認する質問をしてください。

演習6-B 最大値の変化を追う

難易度 ★★☆☆

次の配列を使って、最大値を求める処理をトレースしてください。

```
A ← {2, 5, 1, 9, 8, 10, 7, 3, 6, 4}
```

i	A[i]	比較前の max	A[i] > max	比較後の max
初期化	2	—	—	2
2	5			
3	1			
4	9			
5	8			
6	10			
7	7			
8	3			
9	6			
10	4			

解答欄

最終的な max の値

AI

AIへの質問例 | 自分のトレースを確認するとき

問題6-Bについて質問です。最大値を更新していくトレース表を作成したのですが、max の更新判定や記録内容に間違いがないか見ていただけますか？

演習6-C 添字と値

難易度 ★☆☆

$A = \{8, 3, 6\}$ で、添字は1から始まります。最後の x と A の内容を求めてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← A[2]
A[1] ← x
```

解答欄

最後の x

処理後の配列 A

AI

AIへの質問例 | 考え方や疑問点を伝える

問題6-Cについて質問です。配列の添字(インデックス)と、その場所に格納されている要素の値はどう区別すればよいのでしょうか？

演習6-D 配列を一つずつ読む

難易度 ★☆☆

$A = \{4, 7, 2\}$ 、添字は1から始まります。表示される順番を書いてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
for (i を 1 から 3 まで 1 ずつ増やす)
  表示する(A[i])
endfor
```

解答欄

表示される順番

AI

AIへの質問例 | 考え方や疑問点を伝える

問題6-Dについて質問です。ループを使って配列の要素を先頭から順に表示させるとき、 $A[i]$ の*i*がどう変化していくのか教えてください。

演習6-E 不足している商品を数える

難易度 ★★★

【実践】4商品の在庫 $A = \{2, 8, 0, 5\}$ を点検します。各商品は5個以上が目標です。不足している商品の種類数 $count$ と補充する総個数 $needed$ を正しく計算するように、プログラム中の空欄 $[a]$ 、 $[b]$ に入る適切な組合せを一つ選んでください。添字は1からです。

この問題の変数は整数型です。配列の添字は1から始まります。

```
count ← 0
needed ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if ([ a ])
    count ← [ b ]
    needed ← needed + (5 - A[i])
  endif
endfor
```

選択肢	内容
ア	a: $A[i] < 5$ / b: $count + 1$
イ	a: $A[i] \leq 5$ / b: $count + 1$
ウ	a: $A[i] < 5$ / b: $needed + 1$
エ	a: $A[i] > 5$ / b: $count + 1$

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題6-Eについて質問です。「5個未満の商品」を判定する場合、不等号は「 $<$ 」と「 $\leq$ 」のどちらが適切でしょうか？品目数のカウント方法も教えてください。

演習6-F 最初に一致する商品を探す

難易度 ★★★

【実践】商品番号配列A={7,4,7,9}から目的の値targetを線形探索し、最初に出現した位置(添字i)をpositionに記録します(未発見時は0)。プログラム中の空欄 [a], [b] に入る適切な組合せを一つ選んでください。添字は1からです。

先に確認：探索とは、目的の値を探す処理です。position(ポジション)は位置を覚える変数で、0は「未発見」の目印です。本問は先頭から一つずつ見る方法です。

この問題の変数は整数型です。配列の添字は1から始まります。

```
position ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if ([ a ])
    position ← [ b ]
  endif
endfor
```

選択肢	内容
ア	a: A[i] = target / b: i
イ	a: A[i] = target and position ≠ 0 / b: i
ウ	a: A[i] = target and position = 0 / b: i
エ	a: A[i] = target and position = 0 / b: A[i]

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題6-Fについて質問です。同じ商品番号が複数あるとき、最初の位置を覚えておく考え方が分かりません。

第7章 トレースの技術

7.1 コードを眺めるだけでは解けない

科目Bでは、擬似言語を見て「何となく分かった」と感じて、選択肢を正しく選べないことがあります。値の変化を紙に書き、処理を実行した結果を確認する必要があります。

この作業をトレースといいます。

1回目 $\text{total} \leftarrow 0 + 1$

2回目 $\text{total} \leftarrow 1 + 2$

3回目 $\text{total} \leftarrow 3 + 3$

頭の中だけで追わない

→ 「何回目か」「計算前」「計算後」を表へ書く。トレースは、プログラムの実況中継です。

7.2 最初に表の列を決める

すべての変数を書く必要はありません。次を目安に列を作ります。

いつ？

i

何回目か

何を使う？

$A[i]$

現在の要素

どう変わる？

total

実行前と実行後

- 繰返しを制御する変数
- 条件式で参照する変数
- 処理によって値が更新される変数
- 配列の場合は、添字と現在参照する要素

例

```
整数型: i
整数型: total

total ← 0

for (i を 1 から 3 まで 1 ずつ増やす)
    total ← total + i × 2
endfor
```

i	i × 2	実行前の total	実行後の total
1	2	0	2
2	4	2	6
3	6	6	12

7.3 「いつの値か」を明確にする

次の二つは意味が異なります。

- 文を実行する前の値
- 文を実行した後の値

トレース表の見出しに「実行前」「実行後」と書くと、混乱を防げます。

7.4 条件判定も一行として記録する

選択処理や繰返しでは、処理本体だけでなく条件判定も行われます。

```
i ← 1

while (i ≤ 3)
    表示する(i)
    i ← i + 1
endwhile
```

i が4になったときも、条件 $i \leq 3$ は一度判定されます。ただし、処理本体は実行されません。

i	$i \leq 3$	表示	更新後の i
1	true	1	2
2	true	2	3
3	true	3	4
4	false	—	—

掛け算を繰り返す場合の初期値

値を繰り返し掛ける問題では、合計との初期値の違いにも注意します。足し算は0から始めますが、掛け算は1から始めます。例えば2と3を順に掛けるなら $1 \rightarrow 2 \rightarrow 6$ です。0から始めると何を掛けても0のままになります。

7.5 AIにトレースさせる前に

AIへ「トレース表を作って」と頼むだけでは、AIが作った表を眺めて終わってしまいます。次の順序で使います。

1. 自分で必要な列を決める
2. 最初の一回分を自分で記入する
3. AIには列の過不足、または最初の誤りだけを確認させる
4. 残りを自分で埋める
5. 最後に解説と照合する

AI

AIへの質問例 | 表の設計を確認するとき

トレース表の自作について質問です。私はトレース表の列を「 i 、実行前のtotal、実行後のtotal」と考えました。値の変化を追うために不足している列や、不要な列があれば、その理由を教えてください。

演習7-A 最初の誤りを見つける

難易度 ★★☆☆

次の処理を考えます。

```
整数型: i
整数型: total

total ← 1

for (i を 1 から 4 まで 1 ずつ増やす)
    total ← total × i
endfor
```

ある受講者が次の表を作りました。最初に間違っている行を見つけ、正しい値を記入してください。

i	実行前の total	total × i	実行後の total
1	1	1	1
2	1	2	2
3	2	5	5
4	5	20	20

解答欄

最初に誤っている行(i の値)	
その行の正しい計算結果	
最終的な total の値	

AI

AIへの質問例 | 自分の説明をレビューするとき

問題7-Aについて質問です。私は「○行目の○○が最初の誤り」と考えました。自分の表を添付するので、どの計算を確認し直すべきかヒントをください。

演習7-B 実行前と実行後

難易度 ★☆☆

二行目の実行直前と直後の x を、それぞれ教えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← 3
x ← x + 4
```

解答欄

二行目の実行直前の x

二行目の実行直後の x

AI

AIへの質問例 | 考え方や疑問点を伝える

問題7-Bについて質問です。「 $x \leftarrow x + 4$ 」を実行するとき、右辺の x と左辺の x はそれぞれ代入前と代入後のどちらの値になるのか教えてください。

演習7-C 通らない行を見分ける

難易度 ★☆☆

実行される代入文だけを順に書き、最後のyを求めてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
x ← 2
if (x > 5)
  y ← 10
else
  y ← 20
endif
```

解答欄

実行される代入文

最後の y

AI

AIへの質問例 | 考え方や疑問点を伝える

問題7-Cについて質問です。ifの条件が偽になった場合、else側の処理だけが実行されてif側の処理は飛ばされるという理解で合っていますか？

演習7-D 繰返しの出口を記録する

難易度 ★★☆☆

判定時の*i*、条件の真偽、totalを表にしてください。最後の*i*とtotalはいくつですか。

この問題の変数は整数型です。配列の添字は1から始まります。

```
i ← 1
total ← 0
while (i ≤ 3)
  total ← total + i
  i ← i + 1
endwhile
```

判定時の <i>i</i>	条件 $i \leq 3$ の真偽	判定直後の total
1		
2		
3		
4		

解答欄

最後の *i*

最後の total

AI

AIへの質問例 | 考え方や疑問点を伝える

問題7-Dについて質問です。トレース表を書くとき、最後に条件が「偽」になってループを抜ける行まで記録しなければならないのはなぜですか？

演習7-E 値引き処理の境界を検証する

難易度 ★★★

【実践】一件の注文が1000円以上なら100円引きにします。注文額の配列 $A = \{999, 1000, 1500\}$ の3件を処理するとき、合計金額 $total$ を正しく求めるように、プログラム中の空欄 [a]、[b] に入る適切な組合せを一つ選んでください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
total ← 0
for (i を 1 から 3 まで 1 ずつ増やす)
  pay ← A[i]
  if ([ a ])
    pay ← [ b ]
  endif
  total ← total + pay
endfor
```

選択肢	内容
ア	a: $pay > 1000$ / b: $pay - 100$
イ	a: $pay \geq 1000$ / b: $pay - 100$
ウ	a: $pay \geq 1000$ / b: $total - 100$
エ	a: $pay > 999$ / b: 100

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題7-Eについて質問です。「1000円以上で値引き」の条件を「 > 1000 」にしてしまうと、ちょうど1000円のデータでどんな不都合が起きますか？

演習7-F 連続した記録をまとめる

難易度 ★★★

【実践】同じ番号が連続した部分を一つのまとまり(グループ)として数えます。配列A={2,2,5,5,2}において、前の値と違ったときだけグループ数を増やすように、プログラム中の空欄 [a], [b] に入る適切な組合せを一つ選んでください。

先に確認: A[i - 1]は、一つ前の位置の値です。i=2から始めるので、存在しないA[0]は読みません。≠は「等しくない」です。

この問題の変数は整数型です。配列の添字は1から始まります。

```
groups ← 1
for (i を 2 から 5 まで 1 ずつ増やす)
  if ([ a ])
    groups ← [ b ]
  endif
endfor
```

選択肢	内容
ア	a: A[i] = A[i - 1] / b: groups + 1
イ	a: A[i] ≠ A[i - 1] / b: i
ウ	a: A[i] > A[i - 1] / b: groups + 1
エ	a: A[i] ≠ A[i - 1] / b: groups + 1

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 考え方や疑問点を伝える

問題7-Fについて質問です。同じ番号が続く部分を、どこで一つのまとまりとして区切れればよいかわかりません。

第8章 総合演習

この章では、複数の知識を組み合わせます。最初からAIへ入力せず、問題ごとに次の欄を埋めてください。

問題を解く前の確認欄	
使われている変数	-----
使われている配列	-----
選択処理	-----
繰り返し処理	-----
求めている結果	-----

演習8-A 条件に合う値の件数

難易度 ★★☆☆

配列 **A** には10個の整数が入っています。5以上の値がいくつあるかを数えて表示します。空欄を埋めてください。

```
整数型: i
整数型: count
整数型の配列: A

count ← [ ① ]

for (i を 1 から 10 まで 1 ずつ増やす)
  if ([ ② ])
    [ ③ ]
  endif
endfor

表示する(count)
```

解答欄

空欄 ①

空欄 ②

空欄 ③

AI

AIへの質問例 | 自力で埋めた後

問題8-Aについて質問です。空欄①～③を〇〇と埋めました。各空欄がそれぞれ「初期化」「条件判定」「件数の更新」に対応していると考えたのですが合っていますか？

演習8-B 最大値とその位置

難易度 ★★★

配列 **A** の最大値と、その値が入っている添字を表示します。配列には少なくとも一つの要素があり、添字は1から始まります。最大値が複数ある場合は、最初に現れる位置を表示します。

空欄①～③に入れる内容の組合せとして、適切なものを一つ選んでください。

```
整数型: i, max, maxIndex
整数型の配列: A
max ← A[1]
maxIndex ← 1

for (i を 2 から Aの要素数 まで 1 ずつ増やす)
  if ([ ① ])
    max ← [ ② ]
    maxIndex ← [ ③ ]
  endif
endfor

表示する(max)
表示する(maxIndex)
```

選択肢	内容
ア	① $A[i] > \text{max}$ / ② $A[i]$ / ③ i
イ	① $A[i] \geq \text{max}$ / ② $A[i]$ / ③ i
ウ	① $A[i] < \text{max}$ / ② $A[i]$ / ③ i
エ	① $A[i] > \text{max}$ / ② $A[i]$ / ③ $A[i]$

解答欄

選択肢(ア～エ)

考えるポイント: 条件が $\geq$ の場合の同点時の動作、**max** と **maxIndex** の更新順、繰返し開始位置に注目しましょう。

AI

AIへの質問例 | 条件の違いを考える

問題8-Bについて質問です。 $A[i] > \text{max}$ と $A[i] \geq \text{max}$ の違いが分かりません。最大値が複数ある配列を使って、比較の考え方を説明してください。

演習8-C 記号を正方形に表示する

難易度 ★★★

1以上の整数 `num` が入力されます。`num` 行、各行に `num` 個の `*` を表示し、正方形を作ります。改行せずに文字を表示する処理を 横に表示する()`、`改行する処理を 改行する() `とします。`

空欄①～④に入れる内容の組合せとして、適切なものを一つ選んでください。選択肢の「`i:1～num`」は「`i` を 1 から `num` まで 1 ずつ増やす」の略記です。他の範囲も同じように1ずつ増やし、両端を含みます。

```
整数型: num, i, j
num ← 入力する()

for ([ ① ])
  for ([ ② ])
    [ ③ ]
  endfor
  [ ④ ]
endfor
```

`num` が3の場合の出力例: 1行に `***` を表示し、改行して3行出力します。

選択肢	内容
ア	① <code>i:1～num</code> / ② <code>j:1～i</code> ③ 横に表示する(" <code>*</code> ") / ④ 改行する()
イ	① <code>i:0～num</code> / ② <code>j:1～num</code> ③ 横に表示する(" <code>*</code> ") / ④ 改行する()
ウ	① <code>i:1～num</code> / ② <code>j:1～num</code> ③ 横に表示する(" <code>*</code> ") / ④ 改行する()
エ	① <code>i:1～num</code> / ② <code>j:1～num</code> ③ 改行する() / ④ 横に表示する(" <code>*</code> ")

解答欄

選択肢(ア～エ)

AI

AIへの質問例 | 入れ子の繰返し

問題8-Cについて質問です。二重ループで、外側の繰返しと内側の繰返しはそれぞれ「行数」と「1行の文字数」のどちらを担当するのか教えてください。

演習8-D 条件付きの更新

難易度 ★☆☆

A={2,6}です。最後のresultを教えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
result ← A[1]
if (A[2] > result)
  result ← A[2]
endif
```

解答欄

最後の result

AI

AIへの質問例 | 考え方や疑問点を伝える

問題8-Dについて質問です。「初期値より大きい値が見つかったときだけ更新する」処理で、値が書き換わる流れをどのようにトレースすればよいか教えてください。

演習8-E 二つの値の合計

難易度 ★☆☆

$A = \{3, 5\}$ です。最後のtotalを教えてください。

この問題の変数は整数型です。配列の添字は1から始まります。

```
total ← 0
for (i を 1 から 2 まで 1 ずつ増やす)
  total ← total + A[i]
endfor
```

解答欄

最後の total

AI

AIへの質問例 | 考え方や疑問点を伝える

問題8-Eについて質問です。配列の要素を先頭から順に足して合計(total)を求めるとき、ループの各周でtotalがどう変化するか教えてください。

演習8-F 条件を満たした値だけ合計する

難易度 ★★☆☆

$A = \{3, 8, 5, 1\}$ から5以上の値だけを集計します。count、total、averageを求めてください。averageは実数型です。この配列では対象が必ず一つ以上あります。

整数型: count, total, i 実数型: average 整数型の配列: A

```
count ← 0
total ← 0
for (i を 1 から 4 まで 1 ずつ増やす)
  if (A[i] ≥ 5)
    count ← count + 1
    total ← total + A[i]
  endif
endfor
average ← total ÷ count
```

解答欄

count

total

average

AI

AIへの質問例 | 考え方や疑問点を伝える

問題8-Fについて質問です。条件を満たすデータだけを集計して平均を出すとき、合計を割る値は配列の総要素数ではなく「条件を満たした件数(count)」で合っていますか？

演習後の振り返り

振り返る項目	記入欄
最初に分からなかったこと	
自分で試したこと	
AIへ入力した質問	
AIから得たヒント	
AIの説明を確認した方法	
次に同じ問題を解くときの注意点	

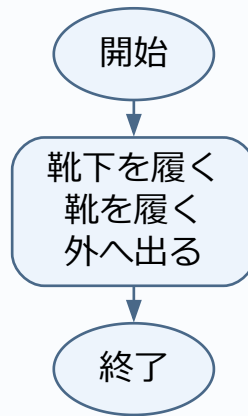
解答・解説

自分の答えを書いてから確認します。文章、値の確認表、フローチャートを対応させて読みましょう。図では読みやすさのため、連続する代入を一つの箱にまとめることがあります。

演習1-A 解答・解説

靴下を履く→靴を履く→外へ出る。上から順に進む処理は「順次」です。逆に戻ったり飛ばしたりせず、書かれた順番に一行ずつ実行します。

演習1-Aのフローチャート

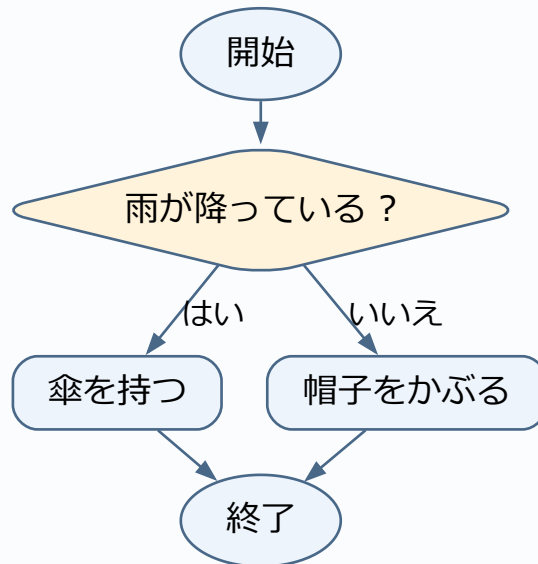


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習1-B 解答・解説

条件で分ける処理は「選択」です。条件は「雨が降っているかどうか」です。条件が「真(成立)」か「偽(不成立)」かで、実行する処理を選びます。

演習1-Bのフローチャート

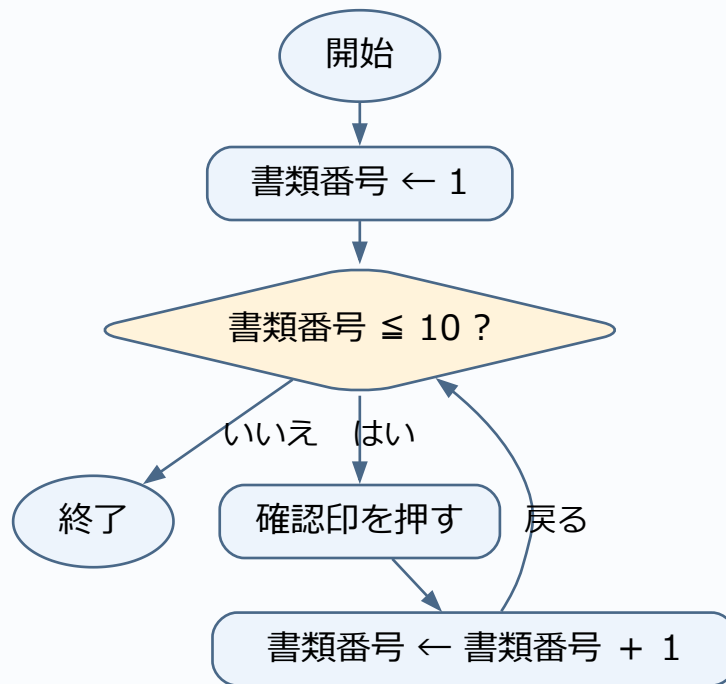


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習1-C 解答・解説

同じ処理を繰り返す「繰り返し」です。終了条件は「10枚すべてに押し終わったとき(残り0枚)」です。回数が決まっている場合と、条件を満たすまで続ける場合があります。

演習1-Cのフローチャート

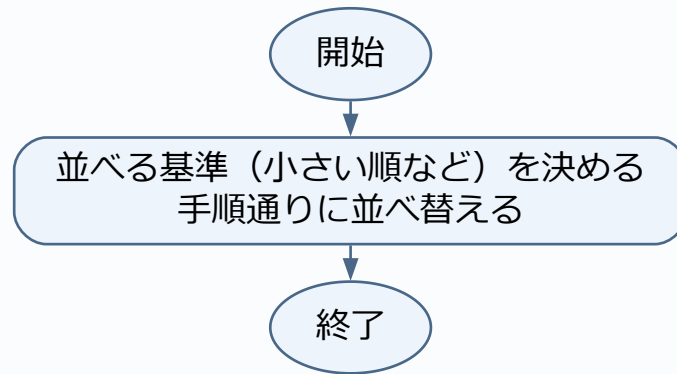


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習1-D 解答・解説

「いい感じ」では基準(小さい順か大きい順か)や終了地点が曖昧だからです。アルゴリズムには明確で誰が見ても同じ結果になる具体的な手順が必要です。

演習1-Dのフローチャート

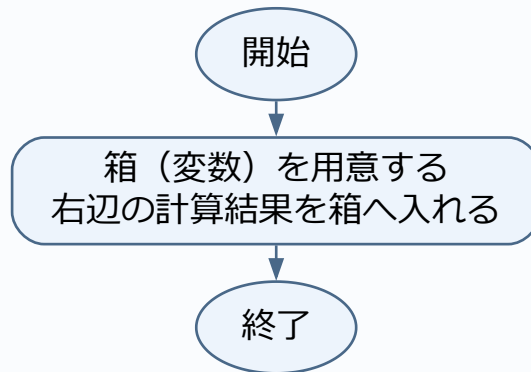


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習1-E 解答・解説

変数は計算結果や入力された値を一時的に覚えておく「名前付きの箱」です。代入「 $\leftarrow$ 」は等しいという意味ではなく、「右辺の計算結果を左辺の箱へ入れる」という動作を表します。

演習1-Eのフローチャート

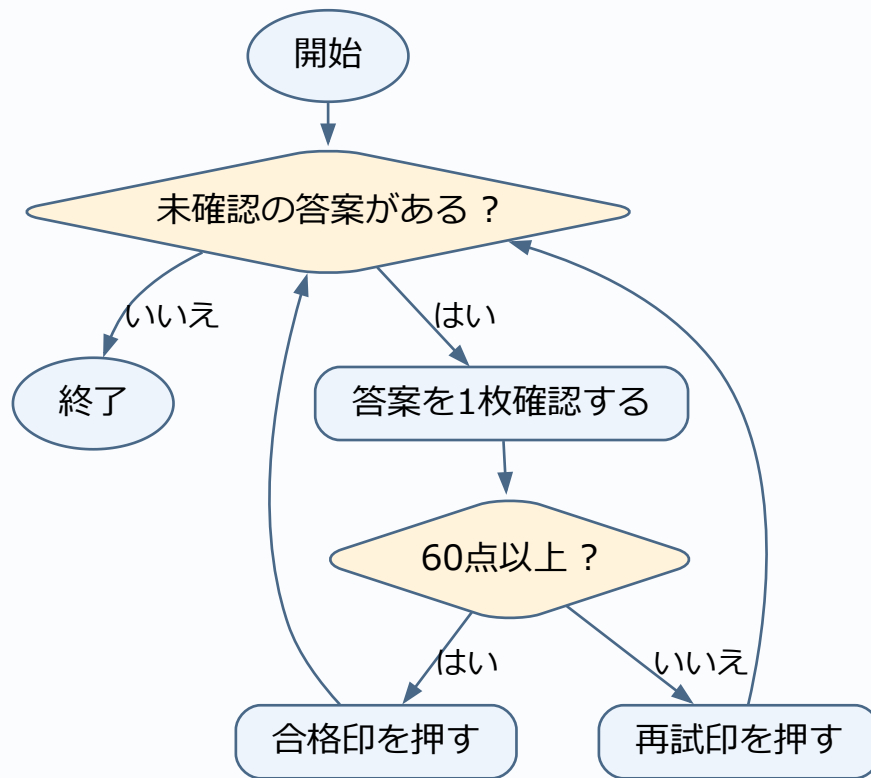


ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習1-F 解答・解説

三つすべてが含まれています。答案を1枚ずつ確認する「繰返し」の中に、点数で印を分ける「選択」があり、一連の作業は上から順に進む「順次」で行われます。

演習1-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

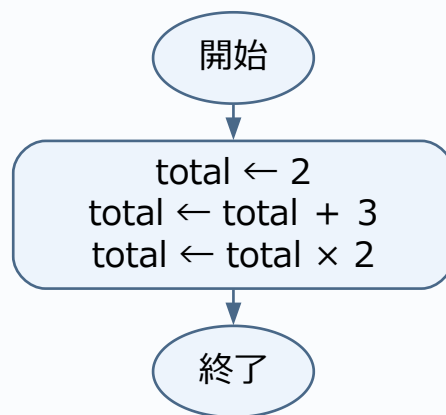
演習2-A 解答・解説

値は2→5→10です。右辺を現在の値で計算してから、左辺を上書きします。

値の確認(各処理の実行後)

実行した処理	その直後の値
<code>total ← 2</code>	<code>total=2</code>
<code>total ← total + 3</code>	<code>total=5</code>
<code>total ← total × 2</code>	<code>total=10</code>

演習2-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

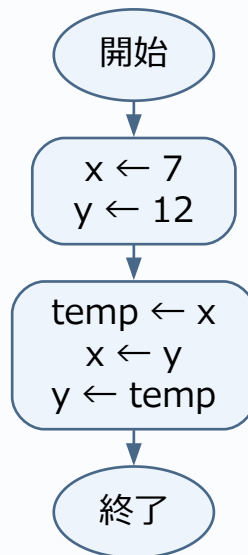
演習2-B 解答・解説

① x、② y、③ temp。tempに元のxを残すので、xを上書きしても7を取り戻せます。最終値はx=12、y=7です。

値の確認(各処理の実行後)

実行した処理	その直後の値
$x \leftarrow 7$	$x=7$
$y \leftarrow 12$	$x=7, y=12$
$\text{temp} \leftarrow x$	$x=7, y=12, \text{temp}=7$
$x \leftarrow y$	$x=12, y=12, \text{temp}=7$
$y \leftarrow \text{temp}$	$x=12, y=7, \text{temp}=7$

演習2-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

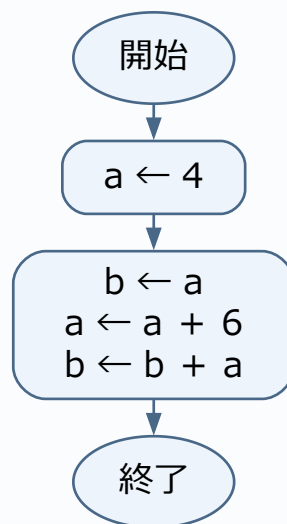
演習2-C 解答・解説

$a=10$ 、 $b=14$ 。 b へコピーした4は、 a を10に変えても自動では変わりません。最後に $4+10$ を計算します。

値の確認(各処理の実行後)

実行した処理	その直後の値
$a \leftarrow 4$	$a=4$
$b \leftarrow a$	$a=4$ 、 $b=4$
$a \leftarrow a + 6$	$a=10$ 、 $b=4$
$b \leftarrow b + a$	$a=10$ 、 $b=14$

演習2-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

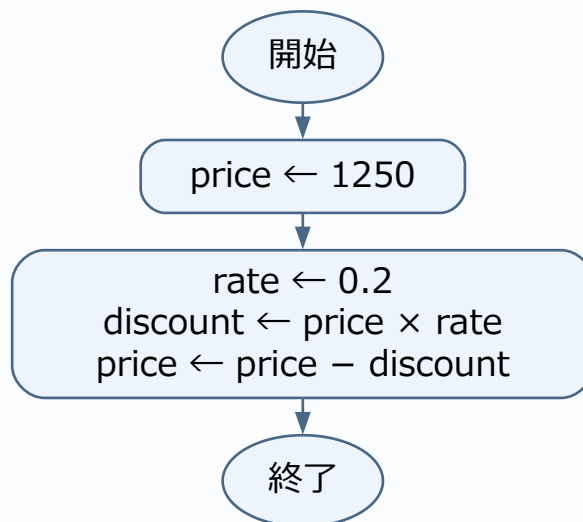
演習2-D 解答・解説

discount=250、price=1000です。0.2を掛けると元の金額の20%になります。割引額と支払金額を区別します。

値の確認(各処理の実行後)

実行した処理	その直後の値
price ← 1250	price=1250
rate ← 0.2	price=1250、rate=0.2
discount ← price × rate	price=1250、rate=0.2、discount=250
price ← price - discount	price=1000、rate=0.2、discount=250

演習2-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

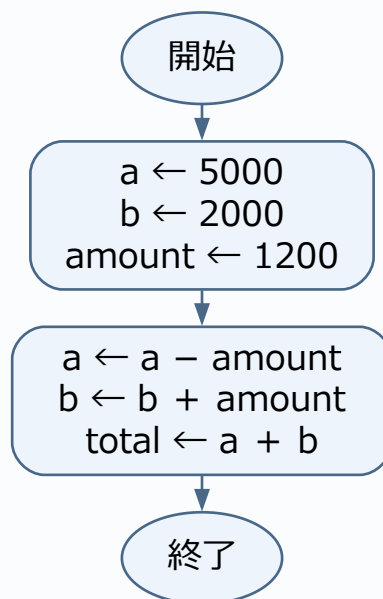
演習2-E 解答・解説

$a=3800$ 、 $b=3200$ 、 $total=7000$ 。口座aから1200を引き、口座bへ1200を足すため、2口座の合計は移動前(7000)と同じです。

値の確認(各処理の実行後)

実行した処理	その直後の値
$a \leftarrow 5000$	$a=5000$
$b \leftarrow 2000$	$a=5000$ 、 $b=2000$
$amount \leftarrow 1200$	$a=5000$ 、 $b=2000$ 、 $amount=1200$
$a \leftarrow a - amount$	$a=3800$ 、 $b=2000$ 、 $amount=1200$
$b \leftarrow b + amount$	$a=3800$ 、 $b=3200$ 、 $amount=1200$
$total \leftarrow a + b$	$a=3800$ 、 $b=3200$ 、 $amount=1200$ 、 $total=7000$

演習2-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

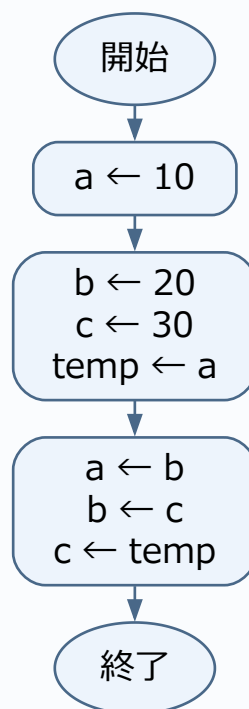
演習2-F 解答・解説

a=20、b=30、c=10です。一時変数tempにaの元値10を退避し、aにbの値20、bにcの値30、cにtempの値10を代入します。

値の確認(各処理の実行後)

実行した処理	その直後の値
a ← 10	a=10
b ← 20	a=10、b=20
c ← 30	a=10、b=20、c=30
temp ← a	a=10、b=20、c=30、temp=10
a ← b	a=20、b=20、c=30、temp=10
b ← c	a=20、b=30、c=30、temp=10
c ← temp	a=20、b=30、c=10、temp=10

演習2-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:変数の退避と上書き。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

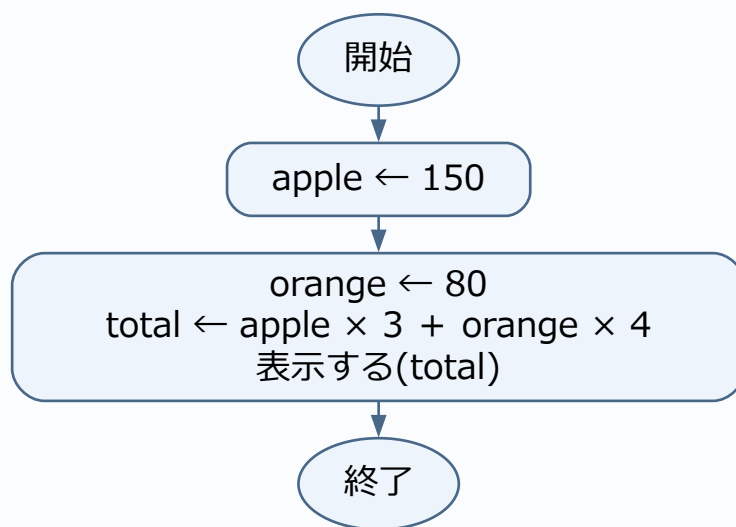
演習3-A 解答・解説

①150、②80、③ $\text{apple} \times 3 + \text{orange} \times 4$ 。450+320=770円です。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{apple} \leftarrow 150$	$\text{apple}=150$
$\text{orange} \leftarrow 80$	$\text{apple}=150, \text{orange}=80$
$\text{total} \leftarrow \text{apple} \times 3 + \text{orange} \times 4$	$\text{apple}=150, \text{orange}=80, \text{total}=770$

演習3-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

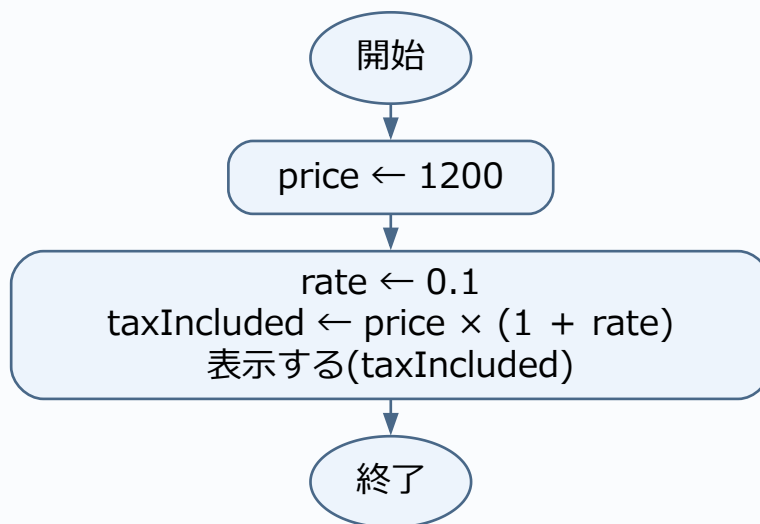
演習3-B 解答・解説

$\text{taxIncluded} \leftarrow \text{price} \times (1 + \text{rate})$ 、表示する(taxIncluded)。税込価格は1320です。税額120だけでなく、本体価格1200も含めます。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{price} \leftarrow 1200$	$\text{price}=1200$
$\text{rate} \leftarrow 0.1$	$\text{price}=1200$ 、 $\text{rate}=0.1$
$\text{taxIncluded} \leftarrow \text{price} \times (1 + \text{rate})$	$\text{price}=1200$ 、 $\text{rate}=0.1$ 、 $\text{taxIncluded}=1320$

演習3-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習3-C 解答・解説

表示は600。入力は単価と個数、処理は掛け算、出力は合計金額の表示です。

値の確認(各処理の実行後)

実行した処理	その直後の値
price ← 200	price = 200
quantity ← 3	price = 200、quantity = 3
total ← price × quantity	price = 200、quantity = 3、total = 600

演習3-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

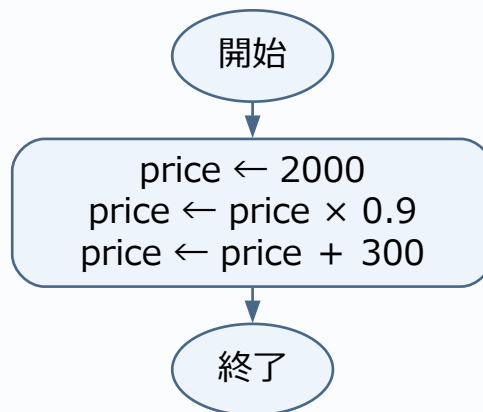
演習3-D 解答・解説

price=2100。先に送料を足すと $2300 \times 0.9 = 2070$ です。後者では送料まで割引対象になるため30円違います。

値の確認(各処理の実行後)

実行した処理	その直後の値
price ← 2000	price=2000
price ← price × 0.9	price=1800
price ← price + 300	price=2100

演習3-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習3-E 解答・解説

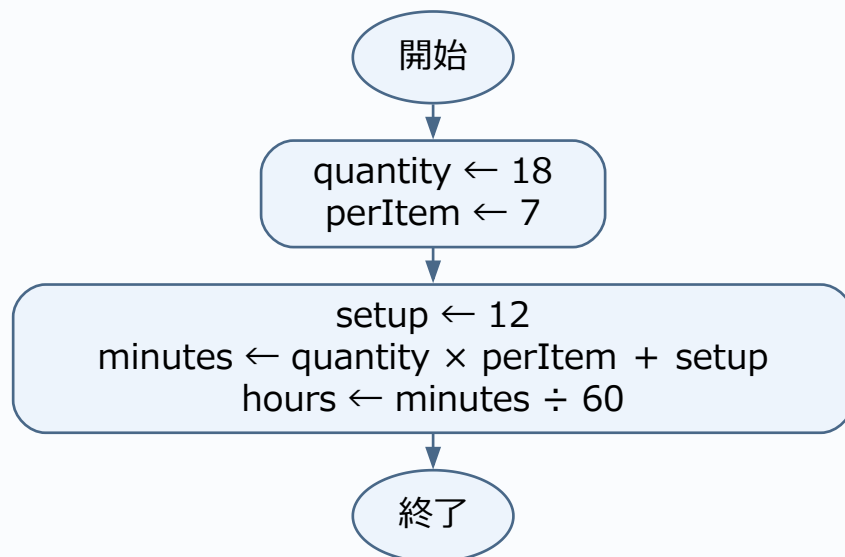
正解:イ

製造時間($\text{quantity} \times \text{perItem}$)に全体の準備時間(setup)を加えるため [a] は $\text{quantity} \times \text{perItem} + \text{setup}$ です。分から時間への換算は60で割るため [b] は $\text{minutes} \div 60$ です($\text{minutes}=138$ 、 $\text{hours}=2.3$)。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{quantity} \leftarrow 18$	$\text{quantity}=18$
$\text{perItem} \leftarrow 7$	$\text{quantity}=18$ 、 $\text{perItem}=7$
$\text{setup} \leftarrow 12$	$\text{quantity}=18$ 、 $\text{perItem}=7$ 、 $\text{setup}=12$
$\text{minutes} \leftarrow \text{quantity} \times \text{perItem} + \text{setup}$	$\text{quantity}=18$ 、 $\text{perItem}=7$ 、 $\text{setup}=12$ 、 $\text{minutes}=138$
$\text{hours} \leftarrow \text{minutes} \div 60$	$\text{quantity}=18$ 、 $\text{perItem}=7$ 、 $\text{setup}=12$ 、 $\text{minutes}=138$ 、 $\text{hours}=2.3$

演習3-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習3-F 解答・解説

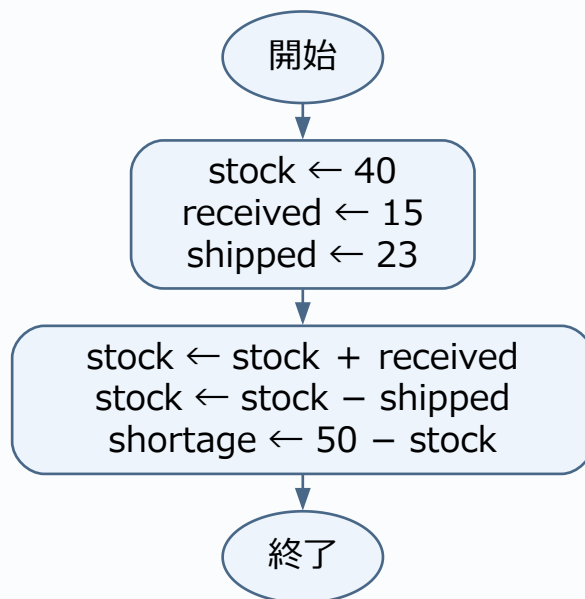
正解:工

入荷時は現在在庫に加算するため [a] は $\text{stock} + \text{received}(55\text{個})$ 、出荷時は減算するため [b] は $\text{stock} - \text{shipped}(32\text{個})$ です。不足数は $50 - 32 = 18\text{個}$ となります。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{stock} \leftarrow 40$	$\text{stock} = 40$
$\text{received} \leftarrow 15$	$\text{stock} = 40, \text{received} = 15$
$\text{shipped} \leftarrow 23$	$\text{stock} = 40, \text{received} = 15, \text{shipped} = 23$
$\text{stock} \leftarrow \text{stock} + \text{received}$	$\text{stock} = 55, \text{received} = 15, \text{shipped} = 23$
$\text{stock} \leftarrow \text{stock} - \text{shipped}$	$\text{stock} = 32, \text{received} = 15, \text{shipped} = 23$
$\text{shortage} \leftarrow 50 - \text{stock}$	$\text{stock} = 32, \text{received} = 15, \text{shipped} = 23, \text{shortage} = 18$

演習3-Fのフローチャート



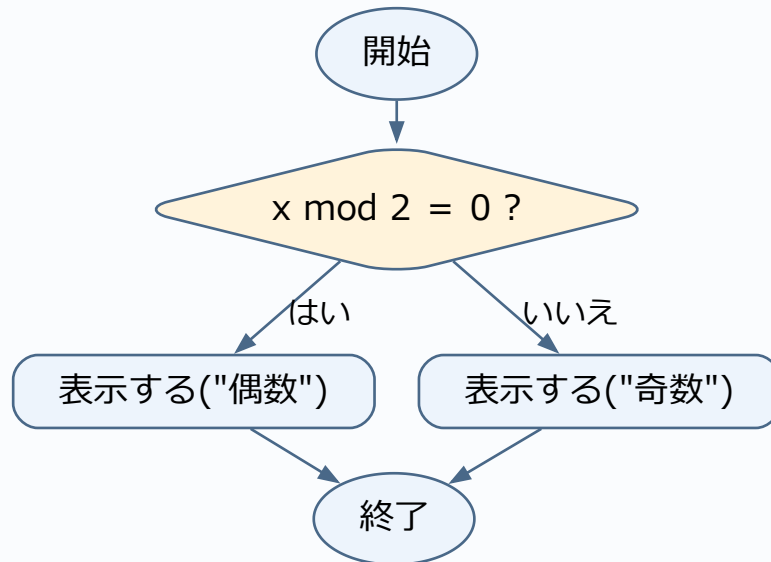
ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習4-A 解答・解説

条件は $x \bmod 2 = 0$ です。余りが0の側だけが偶数になります。 $x=8$ なら偶数、 $x=7$ なら奇数です。

表の確認に使う入力: $x=8$ 。

演習4-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習4-B 解答・解説

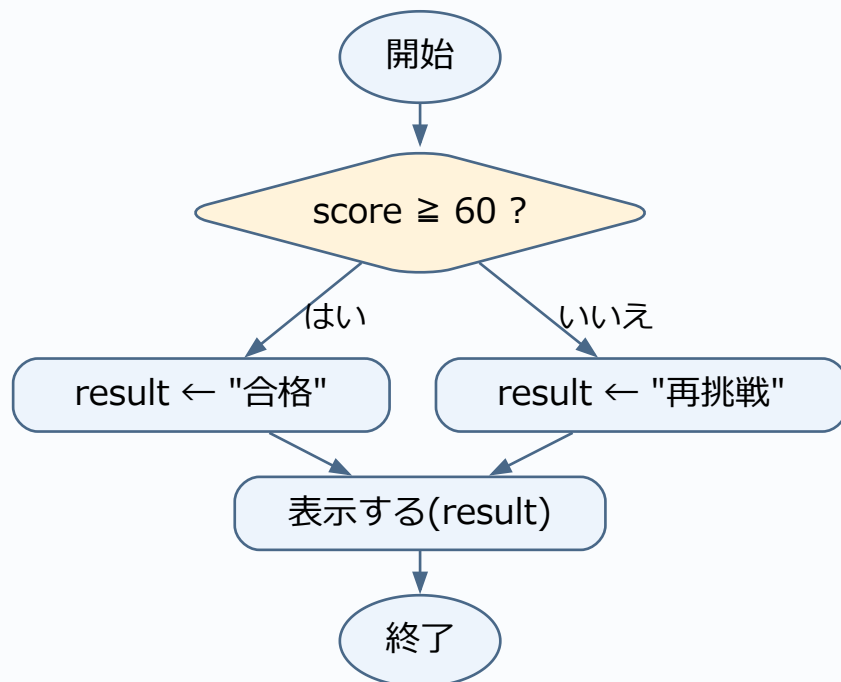
59は再挑戦、60は合格です。「以上」は境目の60自身も含みます。

表の確認に使う入力: score = 59。

値の確認(各処理の実行後)

実行した処理	その直後の値
result ← "再挑戦"	result = 再挑戦

演習4-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習4-C 解答・解説

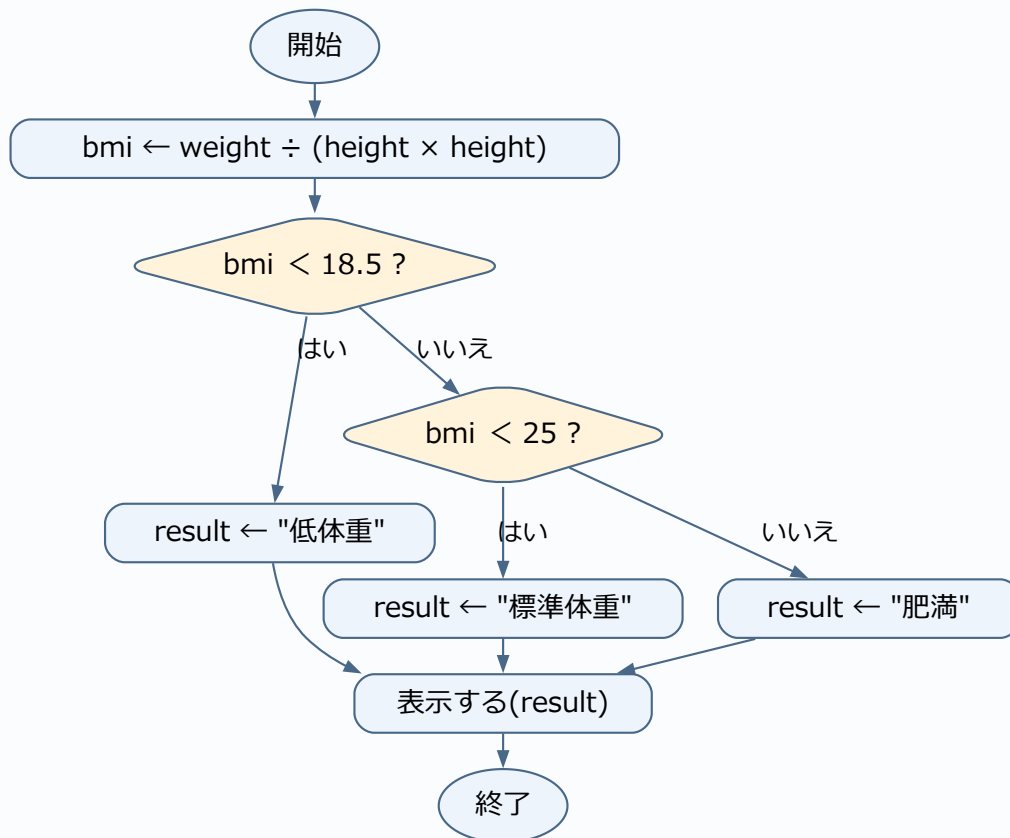
① $\text{bmi} < 18.5$ 、② $\text{bmi} < 25$ 。18.49は低体重、18.50と24.99は標準体重、25.00は肥満です。二つ目へ来た時点で18.5以上は確定しています。

表の確認に使う入力: height=1、weight=18.5。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{bmi} \leftarrow \text{weight} \div (\text{height} \times \text{height})$	$\text{bmi} = 18.5$
$\text{result} \leftarrow \text{"標準体重"}$	$\text{bmi} = 18.5$ 、 $\text{result} = \text{標準体重}$

演習4-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習4-D 解答・解説

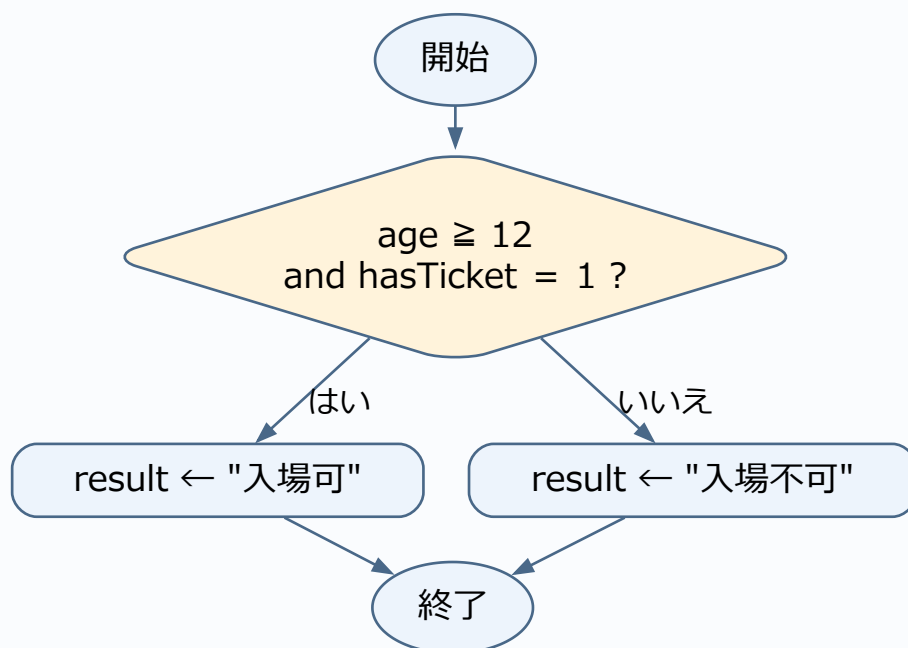
① $\text{age} \geq 12$ and $\text{hasTicket} = 1$ 。結果は順に入場可、入場不可、入場不可です。andは両方が真のときだけ真になります。

表の確認に使う入力: $\text{age}=12$ 、 $\text{hasTicket}=1$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{result} \leftarrow \text{"入場可"}$	$\text{result}=\text{入場可}$

演習4-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習4-E 解答・解説

正解:ア

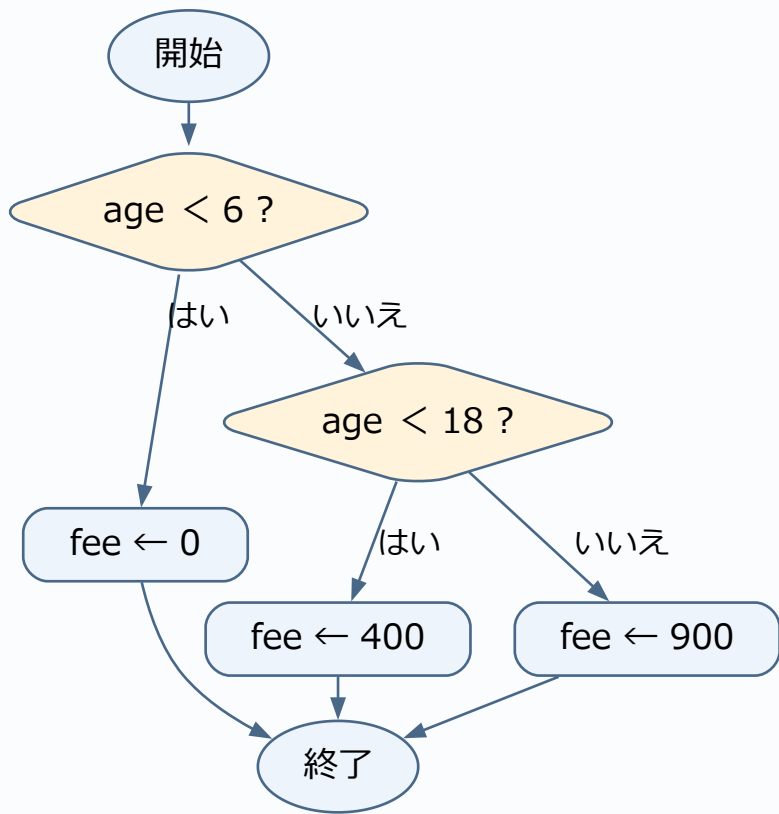
「6歳未満」は6を含まないため [a] は $\text{age} < 6$ です。最初の条件が偽なら6歳以上と確定しているため、続く条件 [b] は $\text{age} < 18$ で6歳以上18歳未満を過不足なく判定できます。

表の確認に使う入力: $\text{age} = 5$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{fee} \leftarrow 0$	$\text{fee} = 0$

演習4-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:条件分岐の読み取り。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

演習4-F 解答・解説

正解:ウ

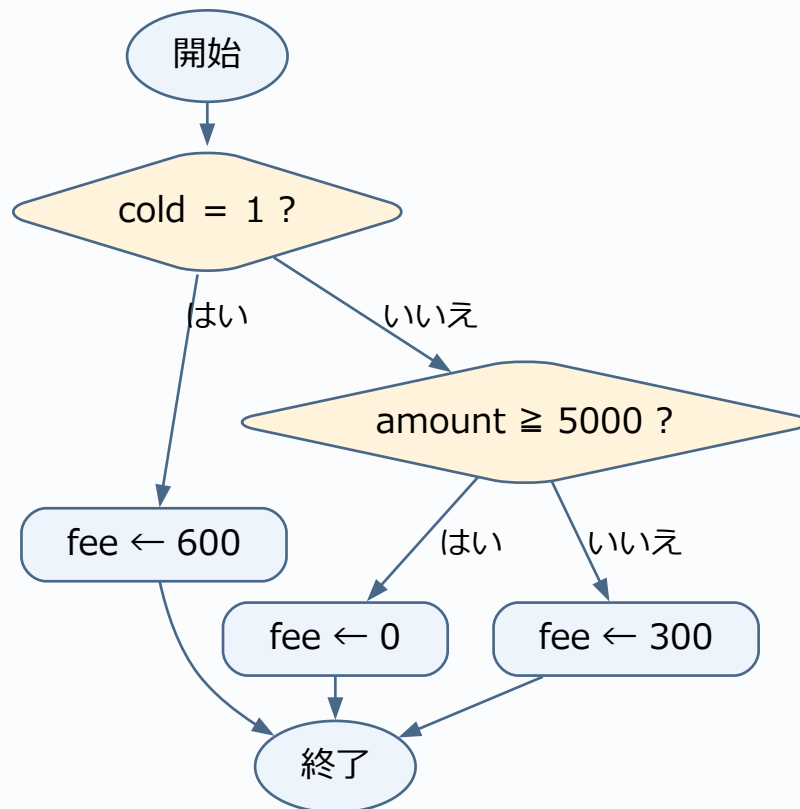
冷蔵条件(cold = 1)を最優先で600円とし、次に通常便の中で5000円以上(amount $\geq$ 5000)を無料と判定します。先に購入額を判定すると冷蔵便まで無料にしてしまうため順序が重要です。

表の確認に使う入力: cold=1、amount=6000。

値の確認(各処理の実行後)

実行した処理	その直後の値
fee $\leftarrow$ 600	fee=600

演習4-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

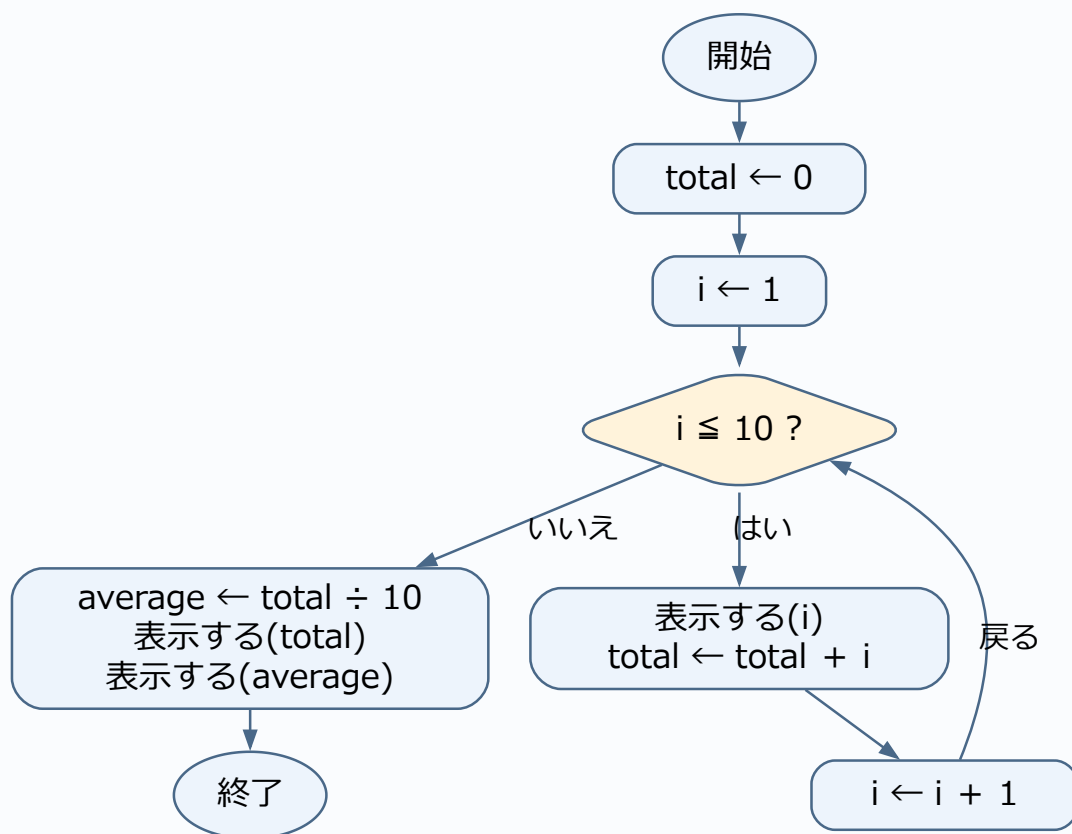
演習5-A 解答・解説

①0、②1、③10、④total + i、⑤total ÷ 10。合計55、平均5.5です。図はforを初期化・条件判定・更新に分解しています。

値の確認(各処理の実行後)

実行した処理	その直後の値
total ← 0	total=0
i=1の回を終了	total=1、i=1
i=2の回を終了	total=3、i=2
…(中間の反復を省略)	
i=9の回を終了	total=45、i=9
i=10の回を終了	total=55、i=10
average ← total ÷ 10	total=55、i=11、average=5.5

演習5-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

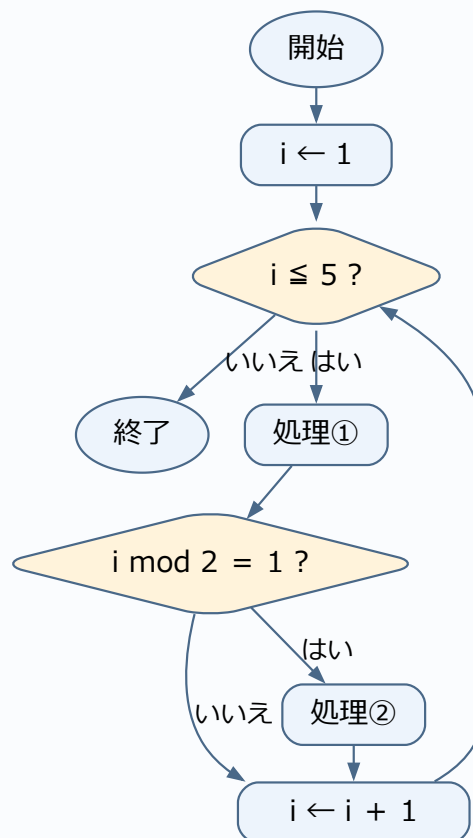
演習5-B 解答・解説

処理①は5回、処理②は $i=1, 3, 5$ の3回。終了時 $i=6$ です。 $i \leq 5$ の判定自体は、最後の偽を含め6回行います。

値の確認(各処理の実行後)

実行した処理	その直後の値
$i \leftarrow 1$	$i=1$
繰返し一回分を終了	$i=2$
繰返し一回分を終了	$i=3$
繰返し一回分を終了	$i=4$
繰返し一回分を終了	$i=5$
繰返し一回分を終了	$i=6$

演習5-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

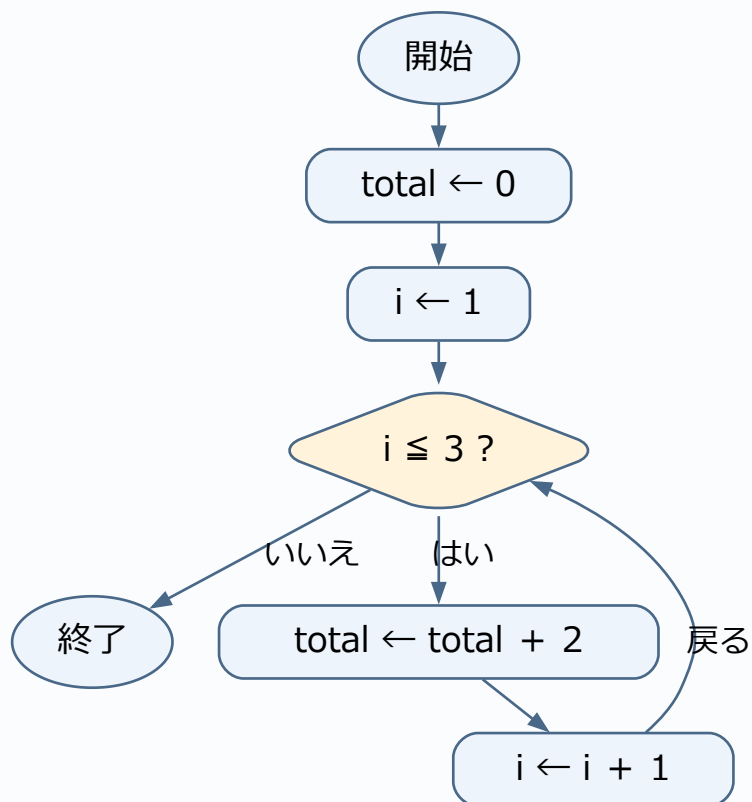
演習5-C 解答・解説

totalは0→2→4→6です。i=1、2、3の計3回、2を足します。

値の確認(各処理の実行後)

実行した処理	その直後の値
total ← 0	total=0
i=1の回を終了	total=2、i=1
i=2の回を終了	total=4、i=2
i=3の回を終了	total=6、i=3

演習5-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

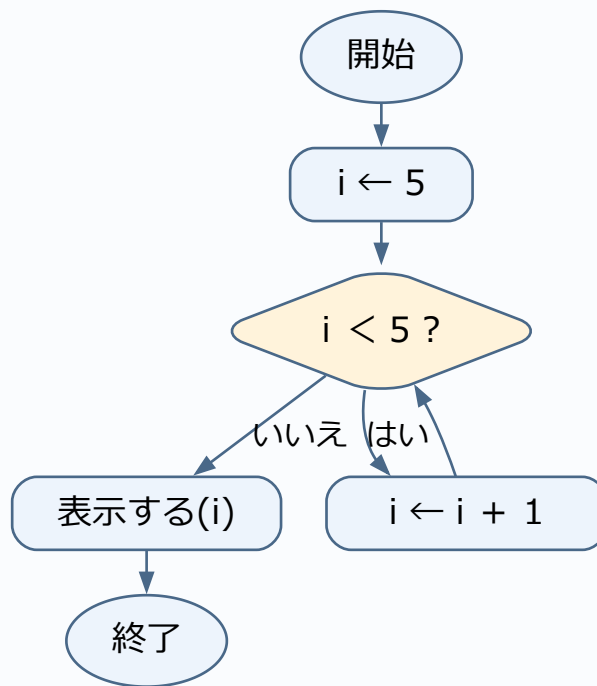
演習5-D 解答・解説

表示は5、内部は0回です。whileは、実行する前に条件を確かめます。最初から偽なら一度も入りません。

値の確認(各処理の実行後)

実行した処理	その直後の値
$i \leftarrow 5$	$i = 5$

演習5-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習5-E 解答・解説

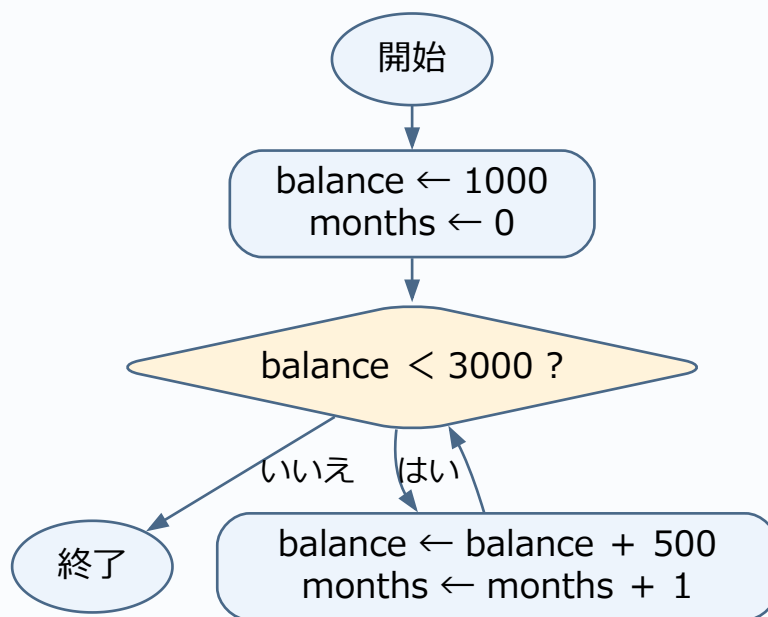
正解:イ

目標の3000円に達するまで(未満の間)繰り返すため [a] は $\text{balance} < 3000$ です。月数を1ずつ加算するため [b] は $\text{months} + 1$ です(終了時: 4か月、3000円)。3000円に達した次の判定は偽になり、そこで止まります。≦にすると3000円でも続行し、5か月・3500円まで積み立ててしまいます。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{balance} \leftarrow 1000$	$\text{balance} = 1000$
繰返し一回分を終了	$\text{balance} = 1500, \text{months} = 1$
繰返し一回分を終了	$\text{balance} = 2000, \text{months} = 2$
繰返し一回分を終了	$\text{balance} = 2500, \text{months} = 3$
繰返し一回分を終了	$\text{balance} = 3000, \text{months} = 4$

演習5-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習5-F 解答・解説

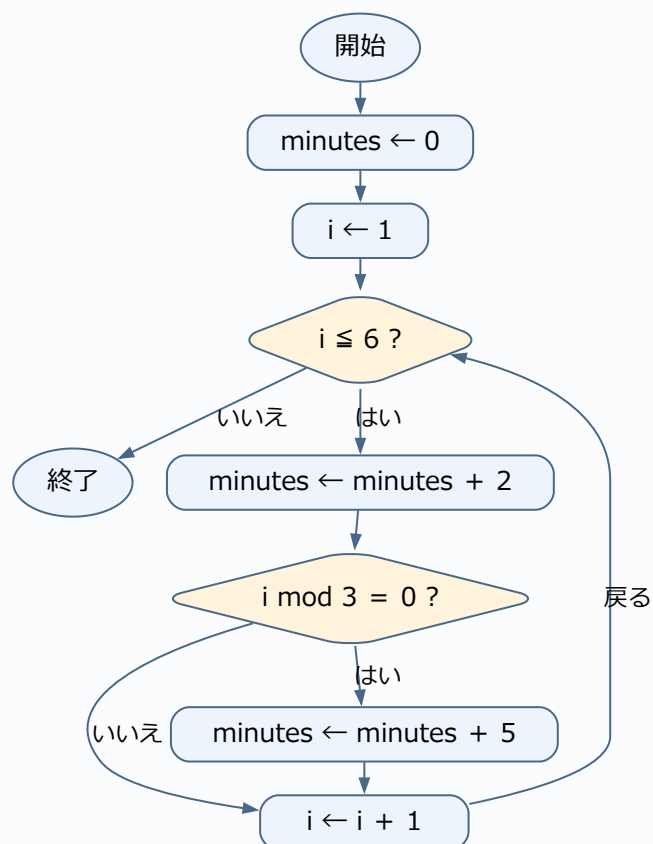
正解:エ

毎回の通常処理時間2分を加算するため [a] は $\text{minutes} + 2$ です。番号が3の倍数の伝票を判定するため [b] は $i \bmod 3 = 0$ です(合計22分)。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{minutes} \leftarrow 0$	$\text{minutes}=0$
$i=1$ の回を終了	$\text{minutes}=2, i=1$
$i=2$ の回を終了	$\text{minutes}=4, i=2$
$i=3$ の回を終了	$\text{minutes}=11, i=3$
$i=4$ の回を終了	$\text{minutes}=13, i=4$
$i=5$ の回を終了	$\text{minutes}=15, i=5$
$i=6$ の回を終了	$\text{minutes}=22, i=6$

演習5-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習6-A 解答・解説

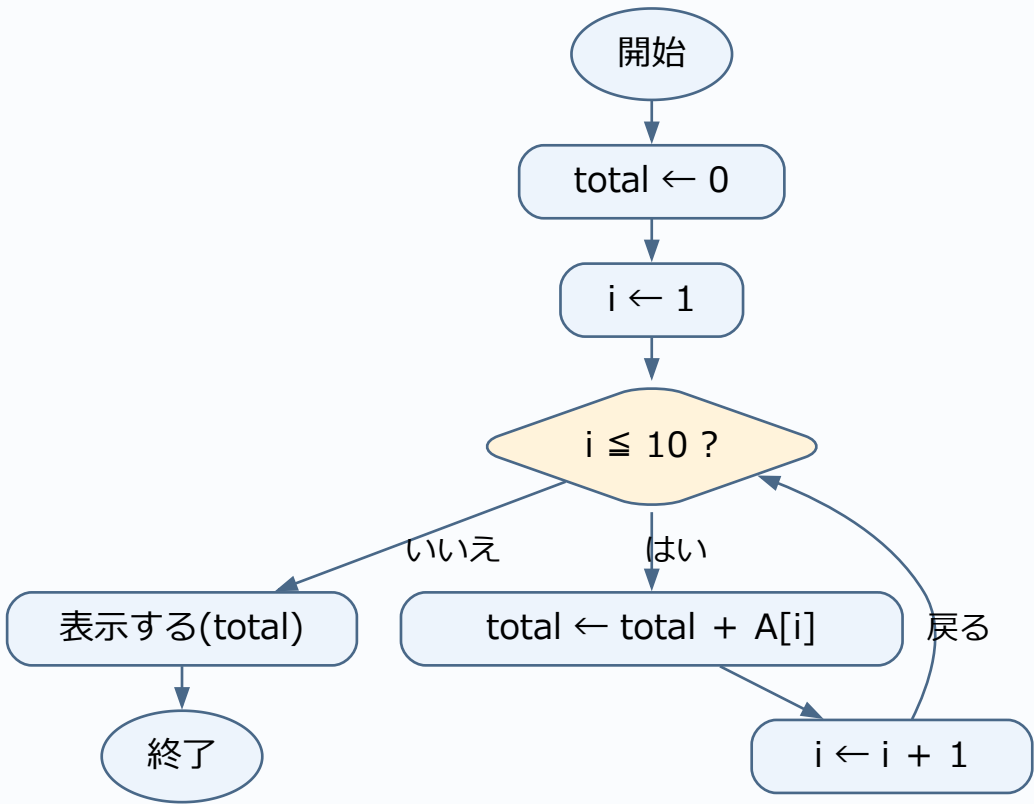
①0、②1、③10、④total + A[i]。iは位置、A[i]はその位置の値です。例えばAが1～10なら合計55。実際の合計は与えられた配列で変わります。

表の確認に使う入力:A={1,2,3,4,5,6,7,8,9,10}。

値の確認(各処理の実行後)

実行した処理	その直後の値
total ← 0	total=0
i=1の回を終了	total=1、i=1
i=2の回を終了	total=3、i=2
...(中間の反復を省略)	
i=8の回を終了	total=36、i=8
i=9の回を終了	total=45、i=9
i=10の回を終了	total=55、i=10

演習6-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習6-B 解答・解説

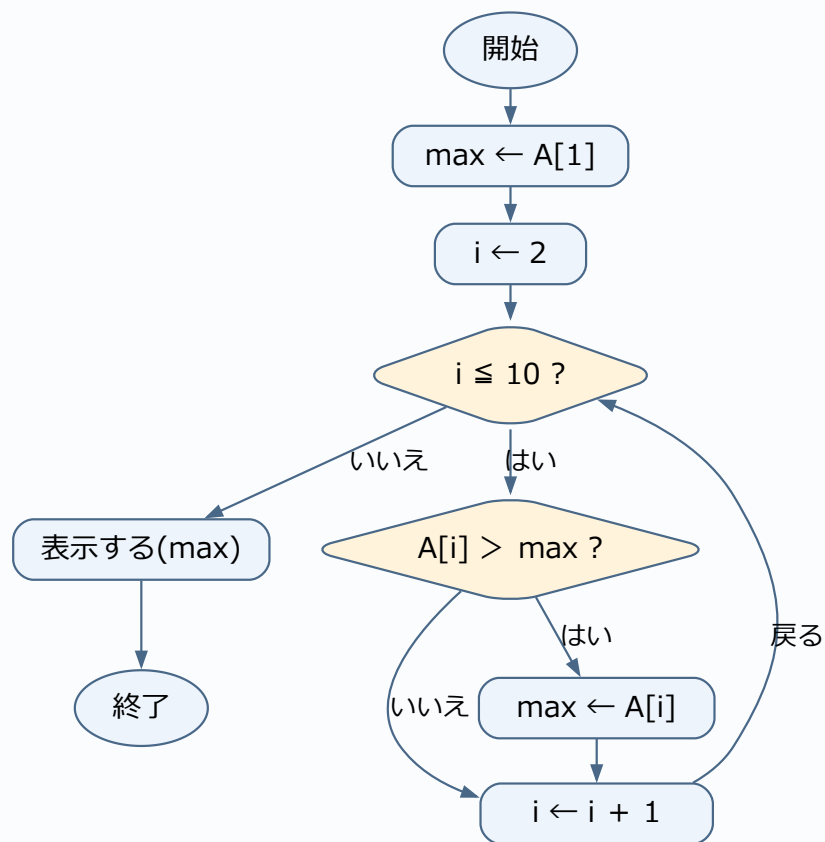
maxは初期値2から、5→5→9→9→10→10→10→10→10。現在の最大値より大きい場合だけ更新します。

表の確認に使う入力: $A = \{2, 5, 1, 9, 8, 10, 7, 3, 6, 4\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{max} \leftarrow A[1]$	$\text{max} = 2$
i=2の回を終了	$\text{max} = 5, i = 2$
i=3の回を終了	$\text{max} = 5, i = 3$
...(中間の反復を省略)	
i=8の回を終了	$\text{max} = 10, i = 8$
i=9の回を終了	$\text{max} = 10, i = 9$
i=10の回を終了	$\text{max} = 10, i = 10$

演習6-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習6-C 解答・解説

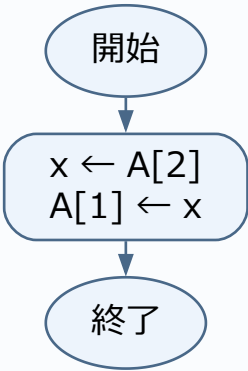
$x=3$ 、 $A=\{3,3,6\}$ です。 $A[2]$ の2は値ではなく位置を指定しています。

表の確認に使う入力: $A=\{8,3,6\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$x \leftarrow A[2]$	$A=\{8, 3, 6\}$ 、 $x=3$
$A[1] \leftarrow x$	$A=\{3, 3, 6\}$ 、 $x=3$

演習6-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習6-D 解答・解説

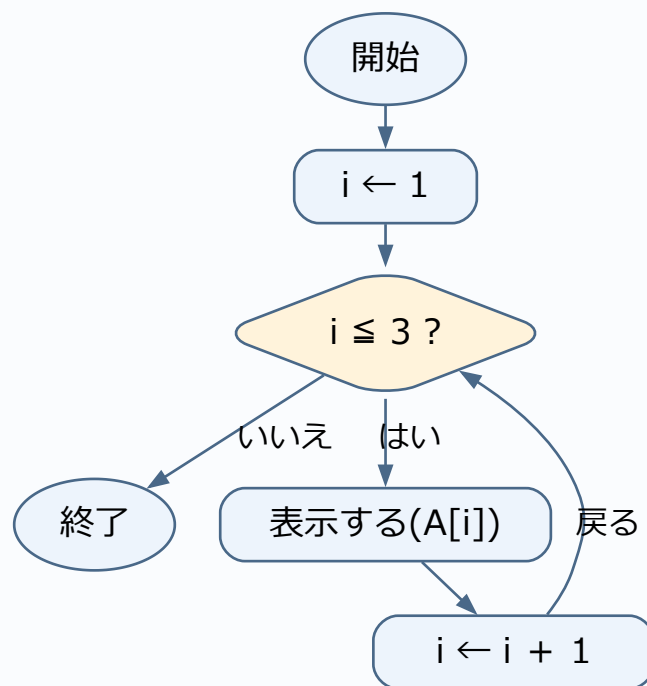
4、7、2の順です。表示するのは添字1、2、3ではなく、その位置に入っている値です。

表の確認に使う入力: $A = \{4, 7, 2\}$ 。

添字と表示される値を確認

i(添字)	A[i](その位置の値)	表示される値
1	4	4
2	7	7
3	2	2

演習6-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習6-E 解答・解説

正解:ア

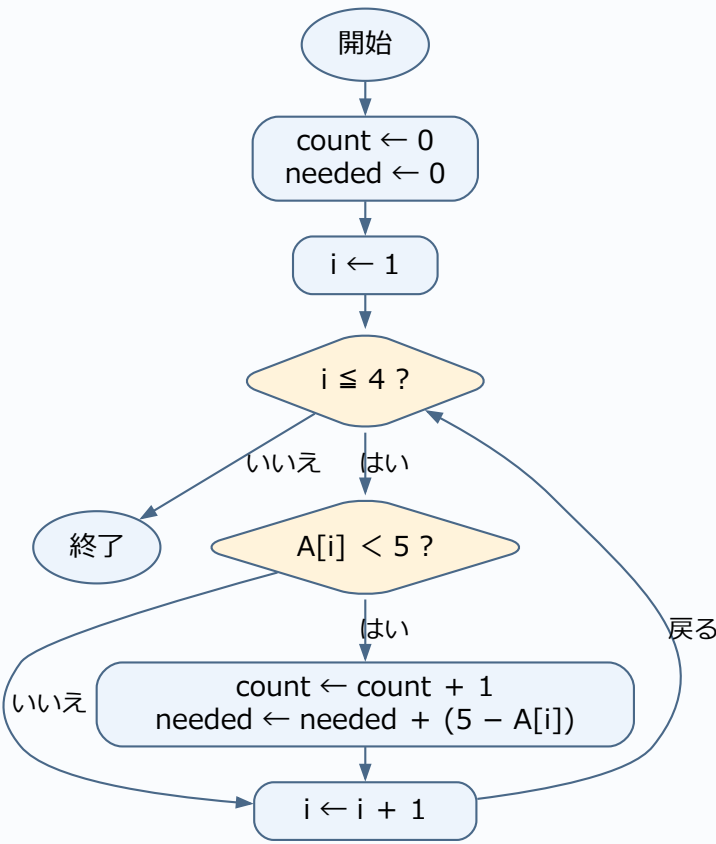
目標の5個未満である商品を判定するため [a] は $A[i] < 5$ です(5個ちょうどの商品は不足していません)。不足商品の種類数を1増やすため [b] は $count + 1$ です($count=2$ 、 $needed=8$)。

表の確認に使う入力: $A = \{2, 8, 0, 5\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$count \leftarrow 0$	$count=0$
$i=1$ の回を終了	$count=1$ 、 $needed=3$ 、 $i=1$
$i=2$ の回を終了	$count=1$ 、 $needed=3$ 、 $i=2$
$i=3$ の回を終了	$count=2$ 、 $needed=8$ 、 $i=3$
$i=4$ の回を終了	$count=2$ 、 $needed=8$ 、 $i=4$

演習6-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習6-F 解答・解説

正解:ウ

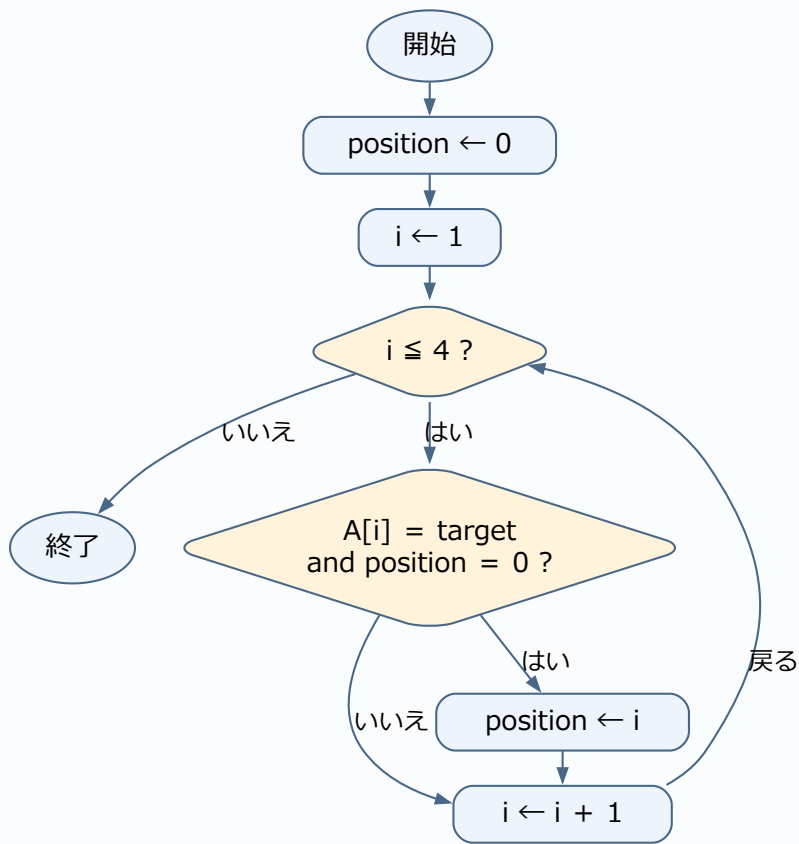
targetと一致し、かつ未発見(position = 0)のときだけ更新することで最初に見つけた位置を保持するため [a] は $A[i] = \text{target}$ and $\text{position} = 0$ です。位置を記録するため [b] は添字の i です。

表の確認に使う入力: $A = \{7, 4, 7, 9\}$ 、target=7。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{position} \leftarrow 0$	$\text{position} = 0$
i=1の回を終了	$\text{position} = 1, i = 1$
i=2の回を終了	$\text{position} = 1, i = 2$
i=3の回を終了	$\text{position} = 1, i = 3$
i=4の回を終了	$\text{position} = 1, i = 4$

演習6-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:探索の考え方(公開問題は二分探索)。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

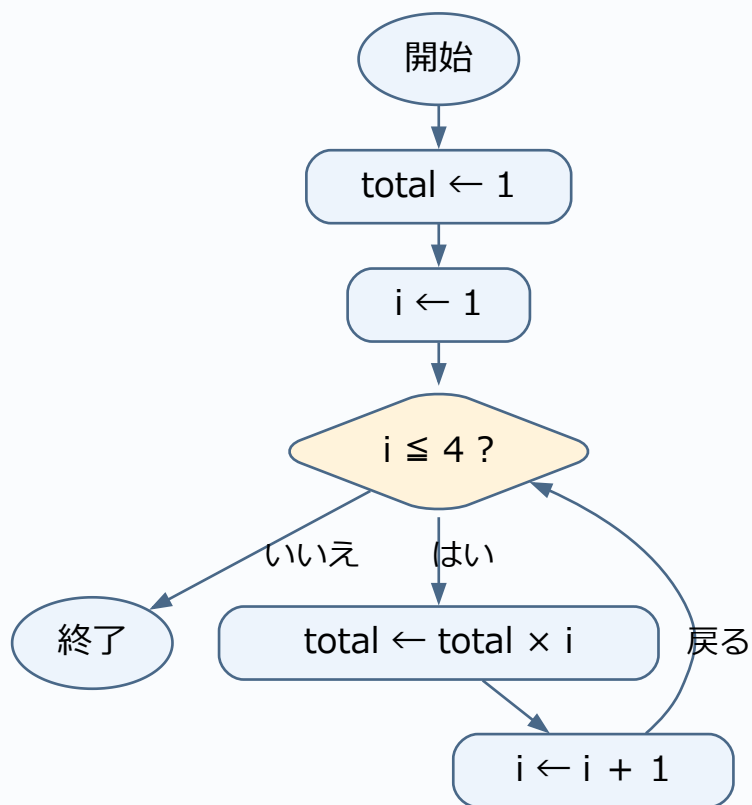
演習7-A 解答・解説

最初の誤りは $i=3$ の行です。 $2 \times 3 = 6$ なので、その行のtotalは6。次の行は $6 \times 4 = 24$ です。誤った5を使い続けな
いよう、以降も直します。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{total} \leftarrow 1$	$\text{total} = 1$
$i=1$ の回を終了	$\text{total} = 1, i = 1$
$i=2$ の回を終了	$\text{total} = 2, i = 2$
$i=3$ の回を終了	$\text{total} = 6, i = 3$
$i=4$ の回を終了	$\text{total} = 24, i = 4$

演習7-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

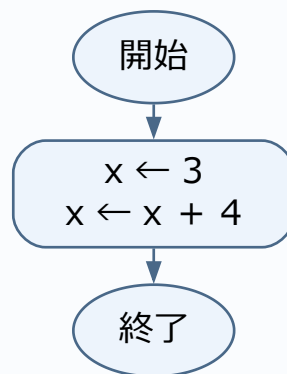
演習7-B 解答・解説

直前は3、直後は7です。右辺で使うxは、代入前の3です。

値の確認(各処理の実行後)

実行した処理	その直後の値
$x \leftarrow 3$	$x=3$
$x \leftarrow x + 4$	$x=7$

演習7-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

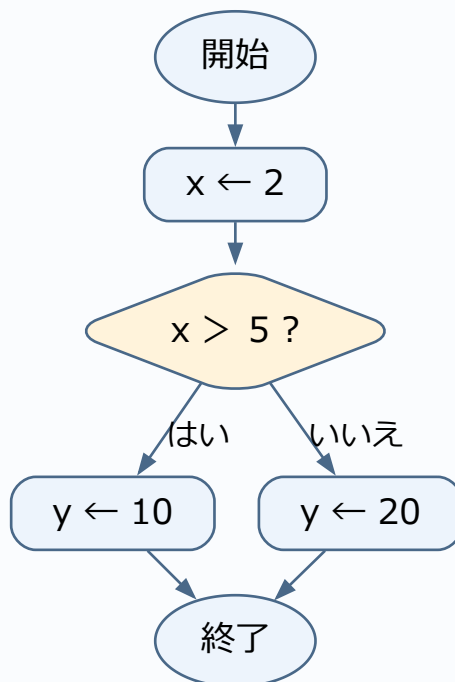
演習7-C 解答・解説

$x \leftarrow 2$ 、 $y \leftarrow 20$ が実行されます。 $2 > 5$ は偽なので $y \leftarrow 10$ は通りません。

値の確認(各処理の実行後)

実行した処理	その直後の値
$x \leftarrow 2$	$x = 2$
$y \leftarrow 20$	$x = 2, y = 20$

演習7-Cのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

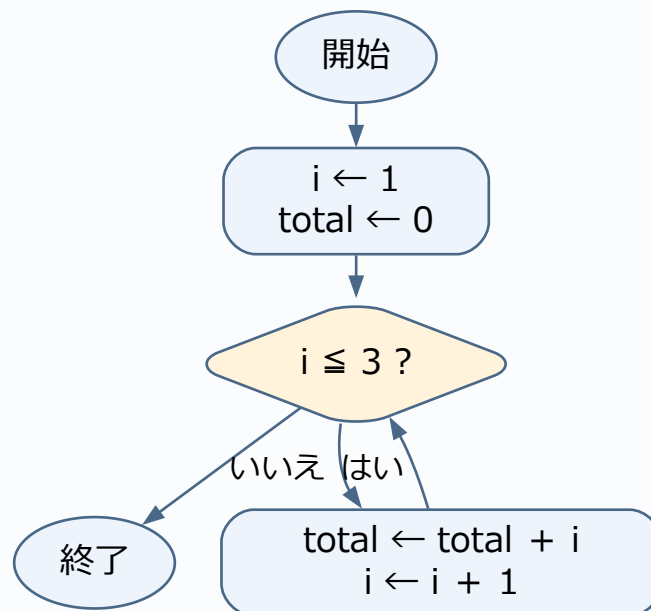
演習7-D 解答・解説

判定時の(i,真偽,total)は(1,真,0)、(2,真,1)、(3,真,3)、(4,偽,6)。終了時 $i=4$ 、 $total=6$ です。偽になった行も記録します。

値の確認(各処理の実行後)

実行した処理	その直後の値
$i \leftarrow 1$	$i=1$
繰返し一回分を終了	$i=2$ 、 $total=1$
繰返し一回分を終了	$i=3$ 、 $total=3$
繰返し一回分を終了	$i=4$ 、 $total=6$

演習7-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習7-E 解答・解説

正解:イ

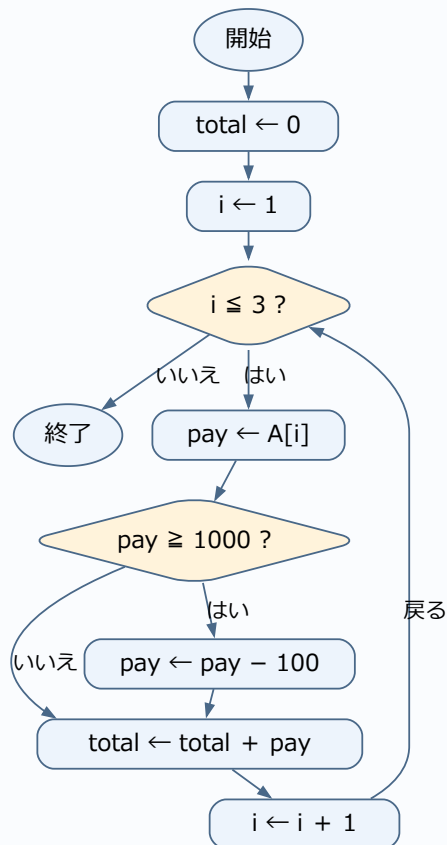
「1000円以上」は1000円自身を含むため条件 [a] は $\text{pay} \geq 1000$ です。 > 1000 にすると1000円の注文が値引きされなくなります。100円引きを行うため [b] は $\text{pay} - 100$ です(合計3299円)。

表の確認に使う入力: $A = \{999, 1000, 1500\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\text{total} \leftarrow 0$	$\text{total} = 0$
i=1の回を終了	$\text{total} = 999, i = 1, \text{pay} = 999$
i=2の回を終了	$\text{total} = 1899, i = 2, \text{pay} = 900$
i=3の回を終了	$\text{total} = 3299, i = 3, \text{pay} = 1400$

演習7-Eのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習7-F 解答・解説

正解:エ

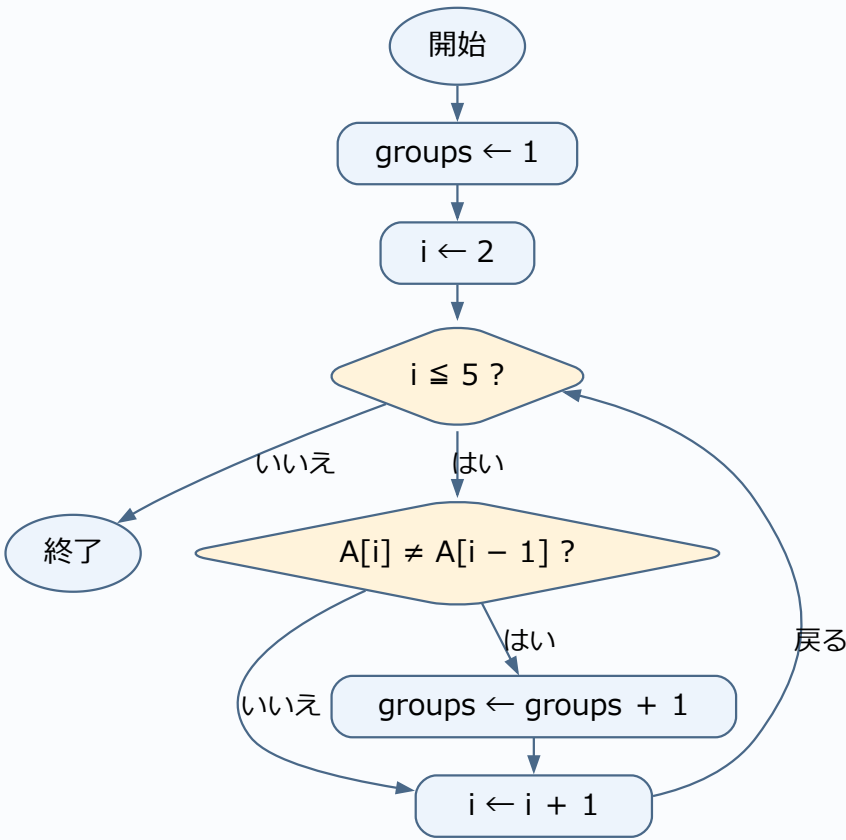
直前の要素と値が異なる場合に新しいグループとなるため条件 [a] は $A[i] \neq A[i - 1]$ です。グループ数を1増やすため [b] は $groups + 1$ です($groups=3$)。

表の確認に使う入力: $A = \{2, 2, 5, 5, 2\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$groups \leftarrow 1$	$groups=1$
$i=2$ の回を終了	$groups=1, i=2$
$i=3$ の回を終了	$groups=2, i=3$
$i=4$ の回を終了	$groups=2, i=4$
$i=5$ の回を終了	$groups=3, i=5$

演習7-Fのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

公開問題への接続:配列を一行ずつ追う練習。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

演習8-A 解答・解説

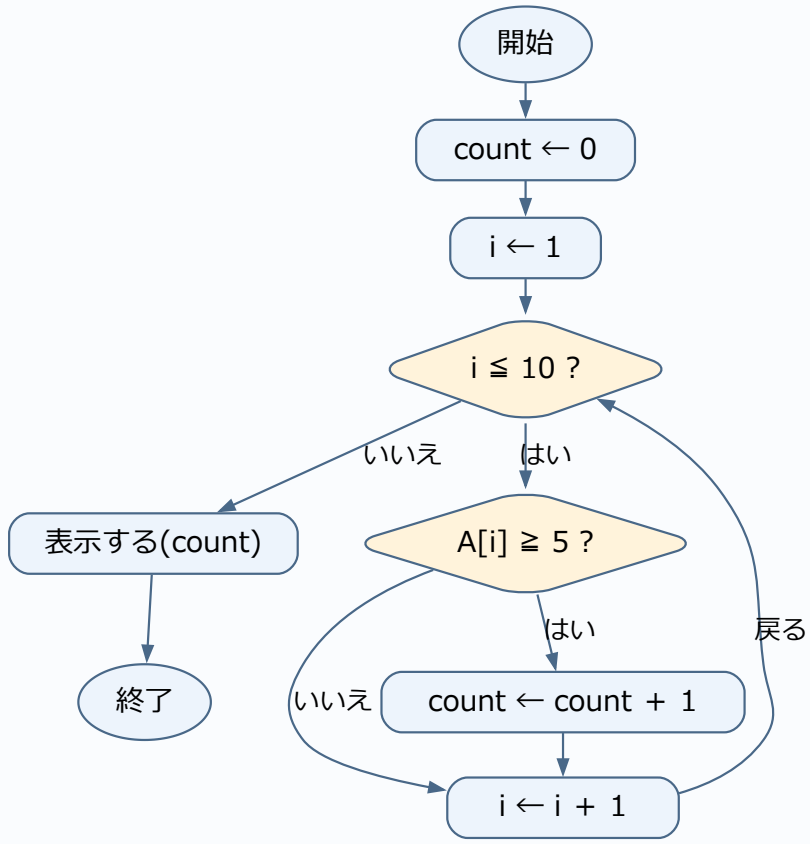
①0、② $A[i] \geq 5$ 、③ $count \leftarrow count + 1$ 。条件が真のときだけ1を足します。5自身も対象です。例えばAが1～10なら6件になります。

表の確認に使う入力: $A=\{1,2,3,4,5,6,7,8,9,10\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$count \leftarrow 0$	$count=0$
$i=1$ の回を終了	$count=0, i=1$
$i=2$ の回を終了	$count=0, i=2$
...(中間の反復を省略)	
$i=8$ の回を終了	$count=4, i=8$
$i=9$ の回を終了	$count=5, i=9$
$i=10$ の回を終了	$count=6, i=10$

演習8-Aのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習8-B 解答・解説

正解:ア

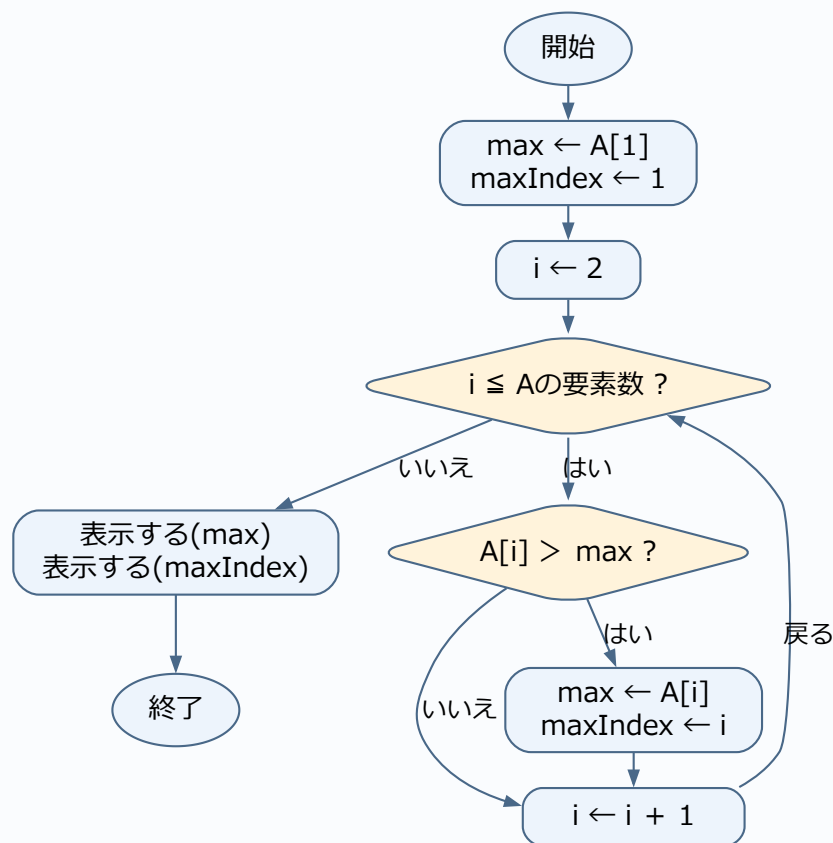
① $A[i] > \max$ 、② $A[i]$ 、③ i 。最大値と位置と一緒に更新します。 $>$ を使うため同点では更新されず、最初の位置が残ります。 $A = \{4, 9, 9\}$ なら $\max = 9$ 、 $\max\text{Index} = 2$ です。

表の確認に使う入力: $A = \{4, 9, 9\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$\max \leftarrow A[1]$	$\max = 4$
$i = 2$ の回を終了	$\max = 9$ 、 $\max\text{Index} = 2$ 、 $i = 2$
$i = 3$ の回を終了	$\max = 9$ 、 $\max\text{Index} = 2$ 、 $i = 3$

演習8-Bのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

公開問題への接続:配列の位置と上書きに注目する練習。本演習は研修用のオリジナル問題であり、リンク先の問題・正解と同一ではありません。

演習8-C 解答・解説

正解:ウ

① i を 1 から num まで 1 ずつ増やす、② j を 1 から num まで 1 ずつ増やす、③ 横に表示する(" $*$ ")、④ 改行する($\backslash n$)。外側が行、内側が一行の文字を担当します。改行は内側を終えてから一回だけです。 $num=3$ なら、各行に $***$ を表示して3行になります。

表の確認に使う入力: $num=3$ 。

$num=3$ の表示結果を確認

外側の <i>i</i>	内側で表示する文字	内側を終えた後
1	***	改行する
2	***	改行する
3	***	改行する

演習8-Cのフローチャート

```
graph TD; Start([開始]) --> I1[i ← 1]; I1 --> ILeqNum{i ≤ num?}; ILeqNum -- いいえ --> End([終了]); ILeqNum -- はい --> J1[j ← 1]; J1 --> JLeqNum{j ≤ num?}; JLeqNum -- はい --> PrintStar[横に表示する("<math>*</math>")]; PrintStar --> JInc[j ← j + 1]; JInc -- 戻る --> JLeqNum; JLeqNum -- いいえ --> PrintNewline[改行する()]; PrintNewline --> IInc[i ← i + 1]; IInc -- 戻る --> ILeqNum;
```

ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。問題で与えられる入力を使い、空欄があれば埋めた処理を示しています。

演習8-D 解答・解説

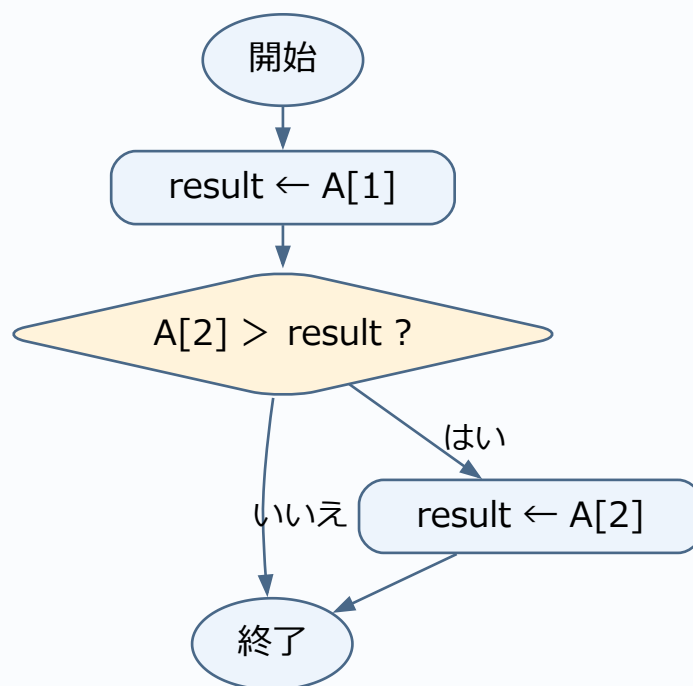
result=6です。初期値2よりA[2]の6が大きいので更新します。

表の確認に使う入力:A={2,6}。

値の確認(各処理の実行後)

実行した処理	その直後の値
result ← A[1]	result=2
result ← A[2]	result=6

演習8-Dのフローチャート



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習8-E 解答・解説

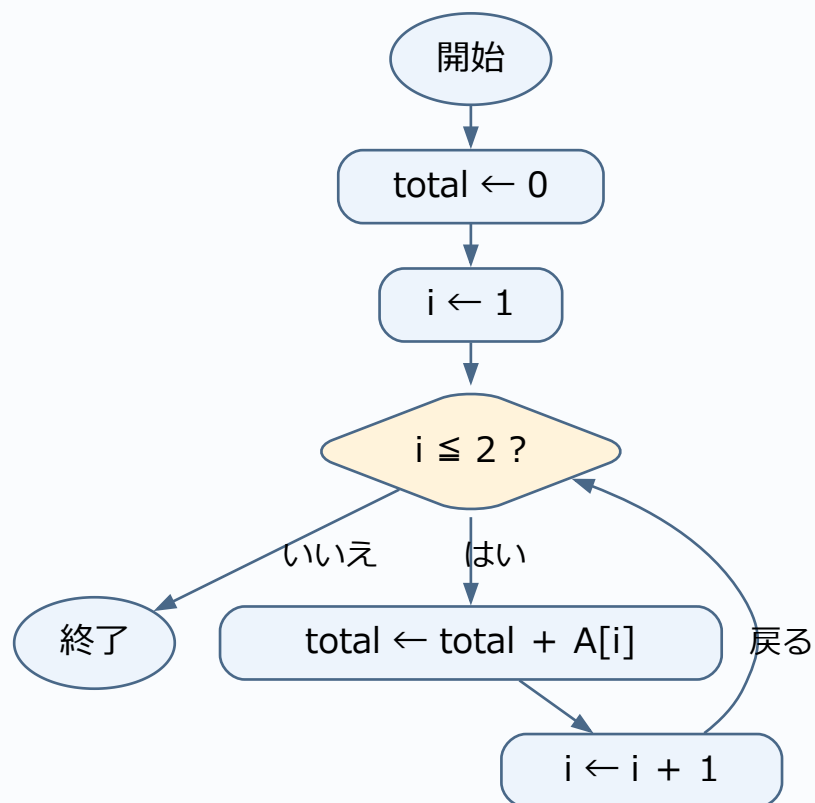
totalは0→3→8です。二つの要素を一回ずつ加算します。

表の確認に使う入力: $A = \{3, 5\}$ 。

値の確認(各処理の実行後)

実行した処理	その直後の値
$total \leftarrow 0$	$total = 0$
$i = 1$ の回を終了	$total = 3, i = 1$
$i = 2$ の回を終了	$total = 8, i = 2$

演習8-Eのフローチャート



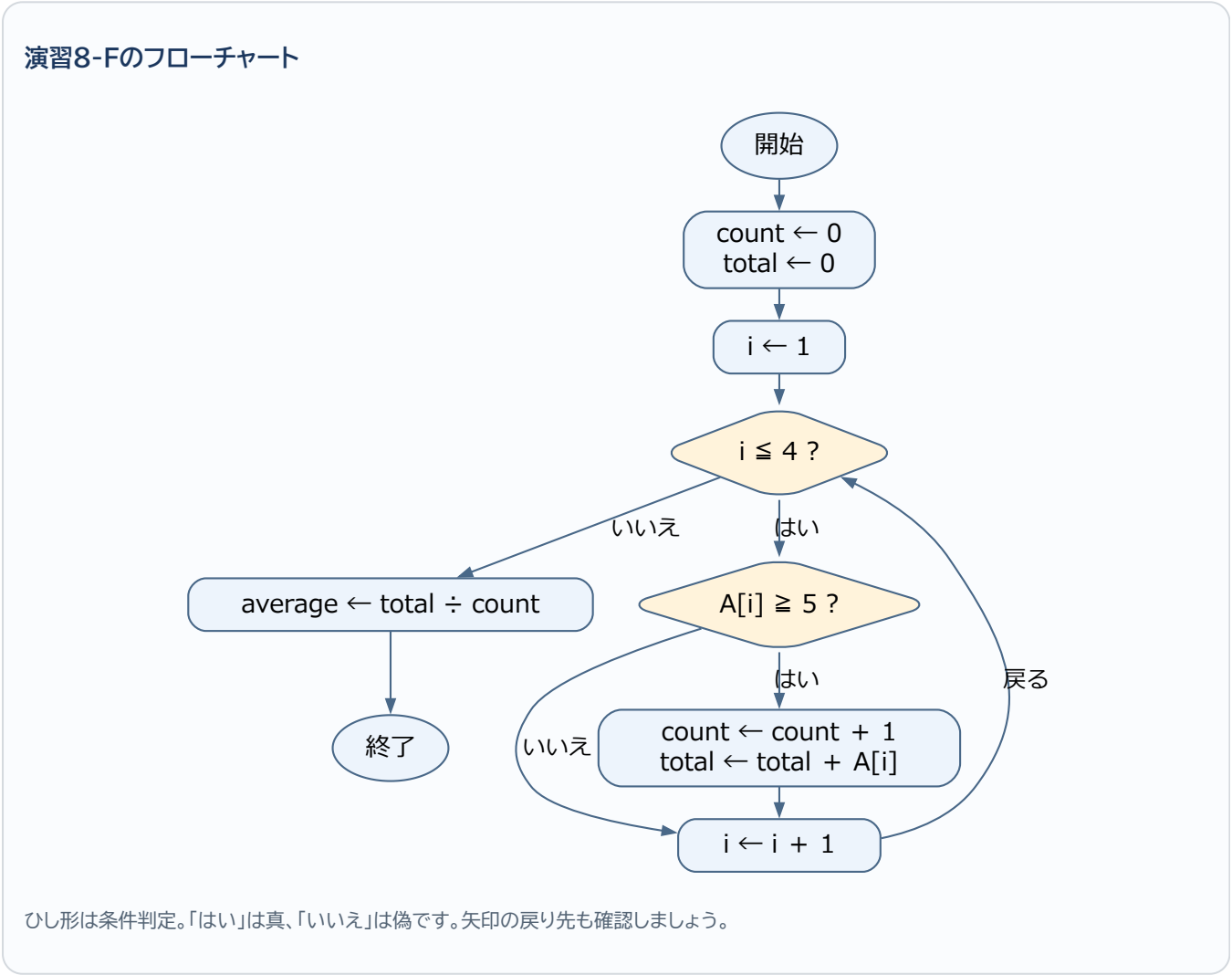
ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

演習8-F 解答・解説

count=2、total=13、average=6.5。対象の8と5だけを足し、配列全体の4個ではなく対象の2個で割ります。
表の確認に使う入力:A={3,8,5,1}。

値の確認(各処理の実行後)

実行した処理	その直後の値
count ← 0	count=0
i=1の回を終了	count=0、total=0、i=1
i=2の回を終了	count=1、total=8、i=2
i=3の回を終了	count=2、total=13、i=3
i=4の回を終了	count=2、total=13、i=4
average ← total ÷ count	count=2、total=13、i=5、average=6.5



ひし形は条件判定。「はい」は真、「いいえ」は偽です。矢印の戻り先も確認しましょう。

研修後の学習ロードマップ

公開問題へ進む前の補助レッスン

公開問題は本教材より長く、まだ学んでいない言葉が登場することがあります。知らない単語だけで止まらず、次の表で読み方を確認します。解けなかった場合は、その知識を補ってから同じ問題へ戻しましょう。

新しい言葉・記法	初心者向けの意味
関数	名前を付けてまとめた処理。例えば「合計を求める」という一まとまり。
引数(ひきすう)	関数へ渡す材料。配列や、探したい値など。
戻り値・return	処理の結果として呼出し元へ返す値。画面への表示とは別。returnに到達すると、その関数の処理を終える。
配列の末尾に追加	最後の後ろに新しい要素を一つ増やす。元の末尾の上書きとは違う。
1ずつ減らすfor	例えばi=5、4、3、2の順で処理する。終了値2も処理する。
昇順	小さい値から大きい値へ並んだ状態。
二分探索	並んだ配列の中央と比べ、探す範囲を狭める方法。単に先頭から探す方法とは違い、並び順が前提となる。

公開問題の読み解き例:配列の要素を移動する

令和8年度 科目B 問1と解説を開き、問題文の「末尾を先頭へ」「それ以外は一つ後ろへ」を確認しましょう。原問題と解答群はリンク先で読みます。以下は同じ処理の考え方を、小さい別の配列で練習するための説明です。

先に確認: len(レン)は要素数、top(トップ)は退避用の変数です。data[i - 1]は移動元、data[i]は移動先です。処理中に値を上書きするので、どちらを先に動かすかが重要です。

data={4,8,2,6}なら、目標は{6,4,8,2}です。まず最後の6をtopへ保存します。その後、iを4→3→2と減らしながら、一つ前の値を移します。

実行した処理	data	top
top ← data[4]	{4,8,2,6}	6
data[4] ← data[3]	{4,8,2,2}	6
data[3] ← data[2]	{4,8,8,2}	6
data[2] ← data[1]	{4,4,8,2}	6
data[1] ← top	{6,4,8,2}	6

逆にiを2から増やすと、最初にdata[2]を4で上書きします。次に読むdata[2]も既に4なので、元の8を移せません。繰返しの方向は「増やすのが普通」と暗記せず、これから読む値を先に消していないかで判断します。原問題に戻り、開始値・終了値・増減の三つが合う選択肢を選びます。

配列が増える公開問題の読み方

サンプル問題 科目B 問3と解説では、配列の末尾への追加と、直前までに求めた値の利用を確認します。読む前に「引数が入力」「戻り値が出力」「末尾に追加すると要素数が1増える」を整理してください。

小さい別例で、入力が{2,4,1}なら、先頭からの合計を順に並べると{2,6,7}になります。2を置く→最後の2に4を足して6を追加→最後の6に1を足して7を追加、という順です。「入力配列」と「作成中の出力配列」を別々に表へ書けば、どの配列の最後を読んでいるか混同しにくくなります。

難しい探索問題は、前提を学んでから取り組む

サンプル問題 科目B 問13と解説は二分探索の不具合を扱います。第6章の「先頭から探す」練習より先の内容です。まず上の用語表を読み、lowは探索範囲の左端、highは右端、middleは中央の位置を表す変数だと整理します。

例えば二つの値{3,9}から9を探すとき、中央を小さい側の位置1とします。見つからなかったのに左端を再び1にすると、範囲は1～2のままです。同じ判定を繰り返して終わりません。「繰返した後に範囲が本当に小さくなったか」を、low・high・middleの表で確認します。講師は二分探索の前提を補足してから取り上げてください。

公開問題を自習するときの質問例：この公開問題の問題文・コードを提示します。まだ二分探索を習っていません。必要な用語と前提を説明し、要素が二つの具体例で一回ずつ処理の流れを教えてください。

公開問題の原文はリンク先で確認します。自習でAIに質問する場合は、第0章0.4の方法で、問題文・コード・選択肢をコピーまたは写真で渡します。本書の演習1-A～8-Fは、研修用の共有ノートブックで問題番号だけを指定して質問できます。

本研修で扱った内容は、科目Bのアルゴリズム問題を読むための入口です。次の順番で学習を続けます。

段階	学習内容	到達目標
1	変数・代入・条件分岐・繰り返し	短いコードをトレースできる
2	一次元・二次元配列	添字と要素を区別できる
3	合計・件数・最大・最小	基本パターンを説明できる
4	線形探索・二分探索	探索範囲の変化を追える
5	基本的な整列	要素の交換を追える
6	スタック・キュー・リスト	データ構造の操作を理解できる
7	再帰・木・グラフ	呼出しと探索順を追える
8	科目B形式の総合問題	時間内に必要な値を追える

自習の一週間モデル

曜日	学習内容
1日目	概念説明を読み、例題をトレースする
2日目	基本問題をAIなしで解く
3日目	間違えた問題についてAIへ質問する
4日目	AIが作成した易しい類題を解く
5日目	本番形式の問題を時間を測って解く
6日目	間違いを「読解・条件・添字・計算・トレース」に分類する
7日目	AIを使わず、間違えた問題を解き直す

研修後に別のFE参考書で自習する場合

自習で新しいチャットを始めるときは、第0章0.5の「手順1」の初回設定を一度送ります。その後は、0.4で紹介したコピー・写真・手入力の方法で問題を渡し、疑問点を質問します。同じチャットでは初回設定を繰り返す必要はありません。

添付した問題について質問です。○行目まで考えましたが、次の行でどの値を使うのか分かりません。私の表を添付するので、確認する場所を教えてください。

本書の演習を復習する場合は、研修用の共有ノートブックで「問題2-Bについて質問です。」のように問題番号を指定します。

最後は必ずAIなしで解く

AIの説明を理解できても、本番でAIは使用できません。各単元の最後には、AIを閉じ、問題文と紙だけで同じ問題をもう一度解いてください。

学習記録

日付	学習範囲	自力で解けた問題	AIへ質問した内容	解き直し
				☐
				☐
				☐
				☐

参考資料

- 基本情報技術者試験.com「科目B 過去問題」(公開問題の所在と解説を確認。教材の48問は研修用オリジナル問題。)
 - Google ヘルプ「ソースの追加」
 - Google ヘルプ「ノートブックの共有」
 - 独立行政法人情報処理推進機構(IPA)「試験要綱・シラバスについて」
 - 独立行政法人情報処理推進機構(IPA)「基本情報技術者試験 科目Bのサンプル問題」
 - 独立行政法人情報処理推進機構(IPA)「令和8年度 基本情報技術者試験 科目B 公開問題」
 - 独立行政法人情報処理推進機構(IPA)「SG・FE 公開問題」
-

本教材の擬似言語は、基本情報技術者試験用の記述形式に合わせています。演習量、難易度、解説の粒度は、研修対象者と実施時間に合わせて調整します。